

REGRESSION ANALYSIS · LOGISTIC REGRESSION

Logistic Regression Models

Sigmoid · maximum likelihood · confusion matrix · ROC and AUC · precision–recall

Hands-on: iris classification and monthly KOSPI direction (Python · scikit-learn · Colab)

Prof. Keehong Rhee

Today in Three Messages

What you should be able to say at the end of this session

1 **Logistic regression is a linear model that outputs probabilities**

A linear combination of features goes through a sigmoid to give a 0–1 probability; coefficients come from maximum likelihood.

2 **The threshold turns probabilities into decisions**

The same model gives different confusion matrices at different thresholds. The ROC curve puts every threshold on one chart.

3 **Choose metrics for the purpose — and the baseline**

Look beyond accuracy: recall, precision, F1, AUC — and always compare with a naive baseline.

Is 55% accuracy good? You cannot say until you know the baseline.

Roadmap

Model → the basic setup for evaluation → confusion matrix → ROC and AUC → labs

#	Section	Key question
01	The logistic regression model	How do we get probabilities, and how are coefficients found?
02	Basic setup for classification metrics	How do probabilities become 'obese / not obese' calls?
03	Confusion matrix and metrics	What did we get right, and what did we get wrong?
04	ROC curve	What changes when we move the threshold?
05	AUC and precision–recall	How do we compare whole models with one number?
06	Basic lab	Fitting and evaluating with scikit-learn
07	KOSPI direction lab	Can we call next month's up or down?

01

SECTION 01

The Logistic Regression Model

Sigmoid · odds and log-odds · maximum likelihood · regularization

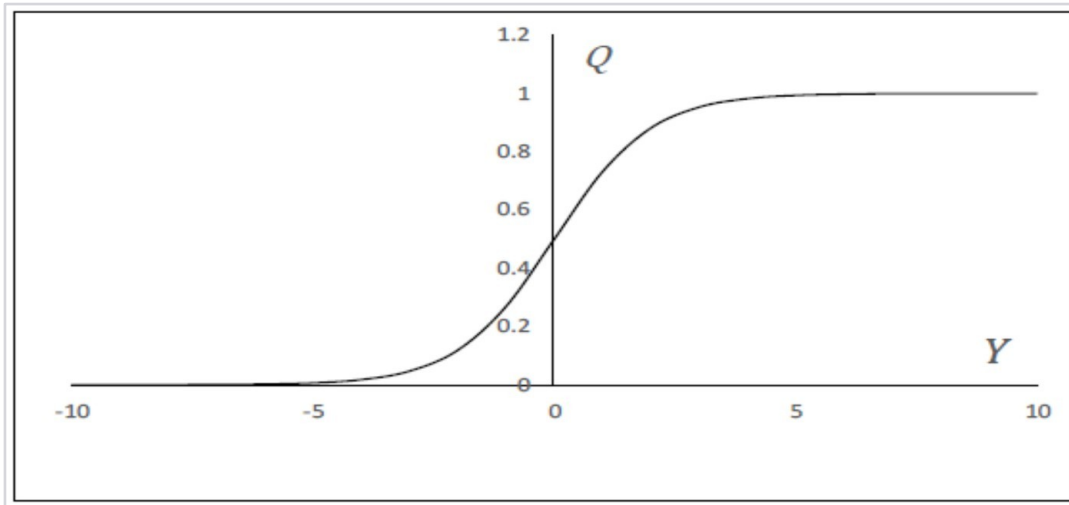
Vocabulary First

Words used throughout classification — pin down the definitions before we start

Term	Meaning	Example
Classification	Assigning each observation to one of a set of classes	obese / not obese, up / down
Positive / negative	Event of interest = positive (1), the rest = negative (0)	obese = 1, not obese = 0
Probability Q	How likely the observation is positive (0–1)	$Q = 0.8$
Odds	$Q / (1 - Q)$: how many times more likely positive than negative	$0.8 / 0.2 = 4$
Log-odds (logit)	$\ln(\text{odds})$ — the quantity logistic regression models linearly	$\ln 4 \approx 1.39$
Threshold	The cut-off that turns a probability into a call	$Q \geq 0.5 \rightarrow \text{positive}$
Decision boundary	Where Q equals the threshold in feature space	$Y = 0$ (threshold 0.5)

Logistic Regression: The Model

Classify observations as positive or negative: feed the linear score Y into a sigmoid to get Q



Fixed from the original: the last term X_m should be $b_m \cdot X_m$

$$Q = P(y = 1 \mid \mathbf{x}) = \frac{1}{1 + e^{-Y}}$$

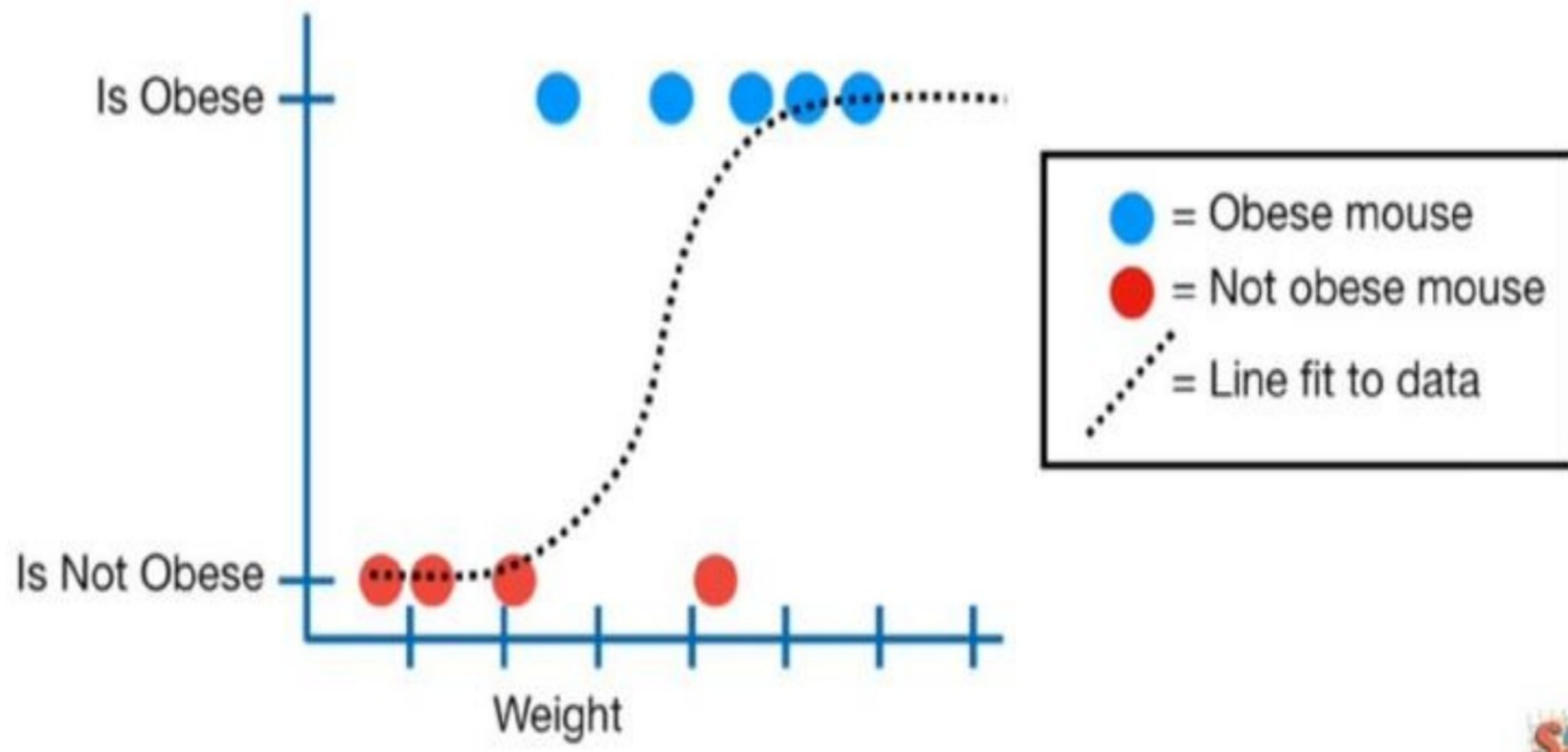
$$Y = a + b_1 X_1 + b_2 X_2 + \cdots + b_m X_m$$

Read it as

- Y can be any real number; Q is always 0–1
- $Y = 0$ gives $Q = 0.5$
- L1 / L2 regularization can be added (sklearn 'l1', 'l2')

Fitting the Data with a Sigmoid

As weight (x) rises, the probability of being obese (blue) climbs from 0 to 1 along an S-curve

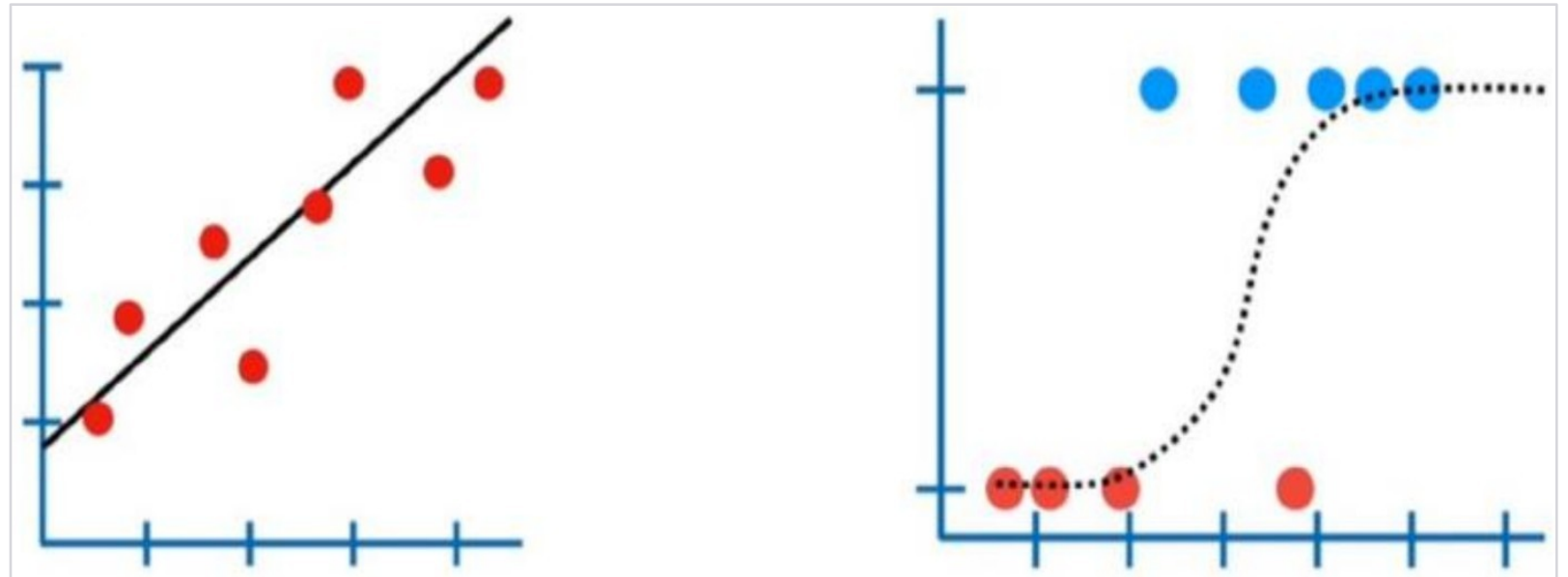


Why Not Linear Regression?

Linear vs. logistic regression — a straight line fitted to 0/1 targets cannot be read as a probability

The difference

- A line runs below 0 and above 1
- One extreme point tilts the slope
- The S-curve stays in 0–1

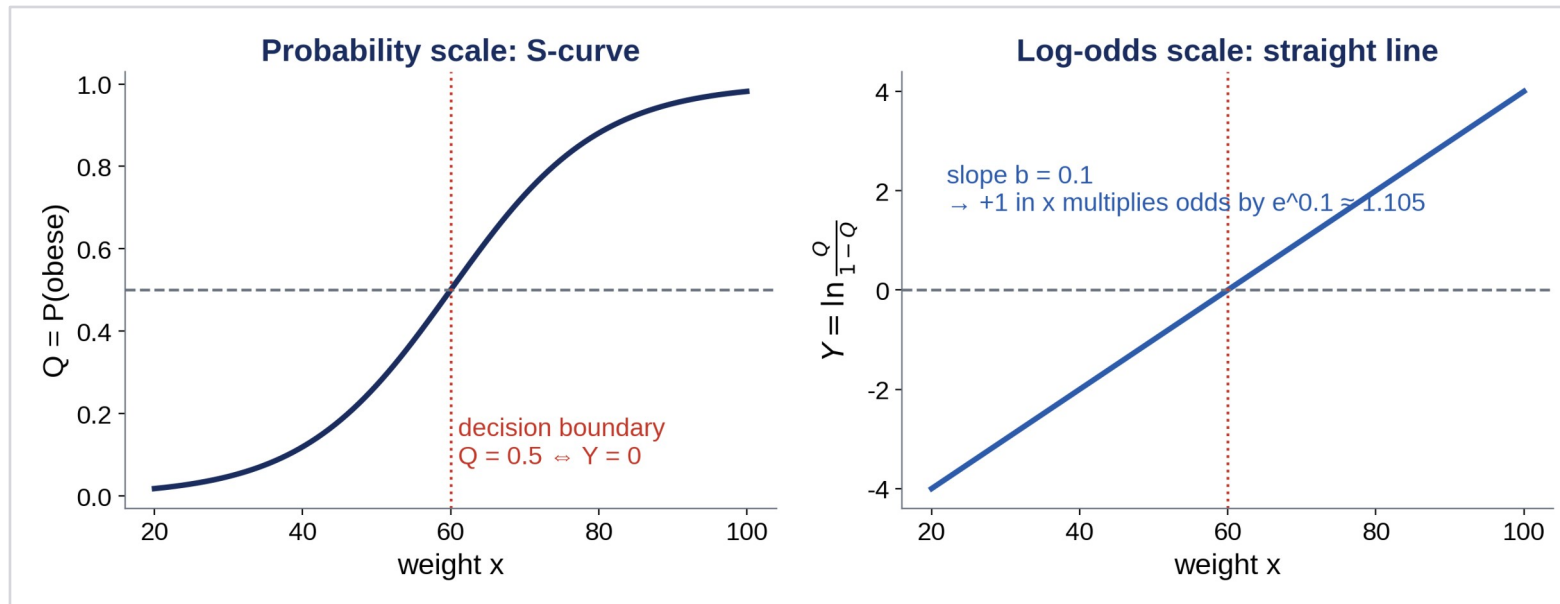


Reading Coefficients: Odds and Log-Odds

Logistic regression makes the log-odds — not the probability — a linear function of the features

$$\ln \frac{Q}{1-Q} = Y = a + b_1 X_1 + \dots + b_m X_m$$

$$\frac{\text{odds}(X_j + 1)}{\text{odds}(X_j)} = e^{b_j}$$



Interpretation

- b_j : change in log-odds when x_j rises by 1
- e^{b_j} : odds multiplier (odds ratio)
- The change in probability depends on the current Q

Maximum Likelihood Estimation

Find coefficients that make Q large for positives and $1 - Q$ large for negatives in the training set

$$\max_{a, \mathbf{b}} \sum_{\text{positive}} \ln Q_i + \sum_{\text{negative}} \ln(1 - Q_i)$$

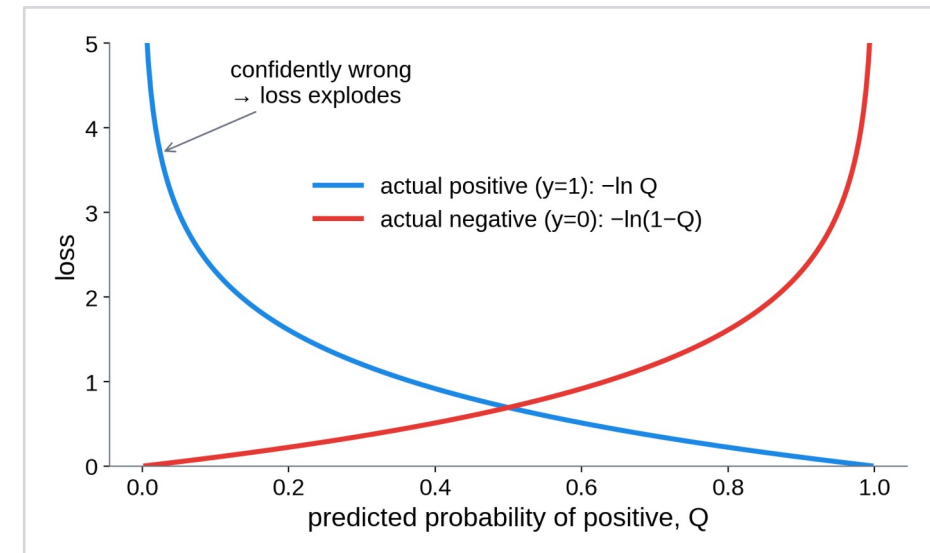
maximize the log-likelihood

$$J = -\frac{1}{m} \sum_{i=1}^m \left[y_i \ln Q_i + (1 - y_i) \ln(1 - Q_i) \right]$$

= minimize log loss (cross-entropy)

How it is solved

- No closed form — solved iteratively
- Gradient ascent on the log-likelihood = gradient descent on log loss
- sklearn uses lbfgs, liblinear, saga, ...



Regularization in scikit-learn: C Is the Inverse

Same L2 / L1 penalties as Ridge and Lasso — but the knob C turns the other way

$$\text{L2 : } \min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|_2^2 + C \sum_{i=1}^m \ell_i$$

$$\text{L1 : } \min_{\mathbf{w}, b} \|\mathbf{w}\|_1 + C \sum_{i=1}^m \ell_i \quad C = \frac{1}{\lambda}$$

Reading C

- $C = 1/\lambda$: smaller C = stronger regularization
- Opposite direction to alpha in linear models
- Defaults: $C = 1.0$, `penalty = 'l2'`
- L1 sets coefficients to 0 → feature selection

Practical notes

- Penalties are scale-sensitive → standardize
- L1 needs the liblinear or saga solver
- From scikit-learn 1.8: `l1_ratio` replaces `penalty` (1 = L1, 0 = L2)
- Choose C by cross-validation

02

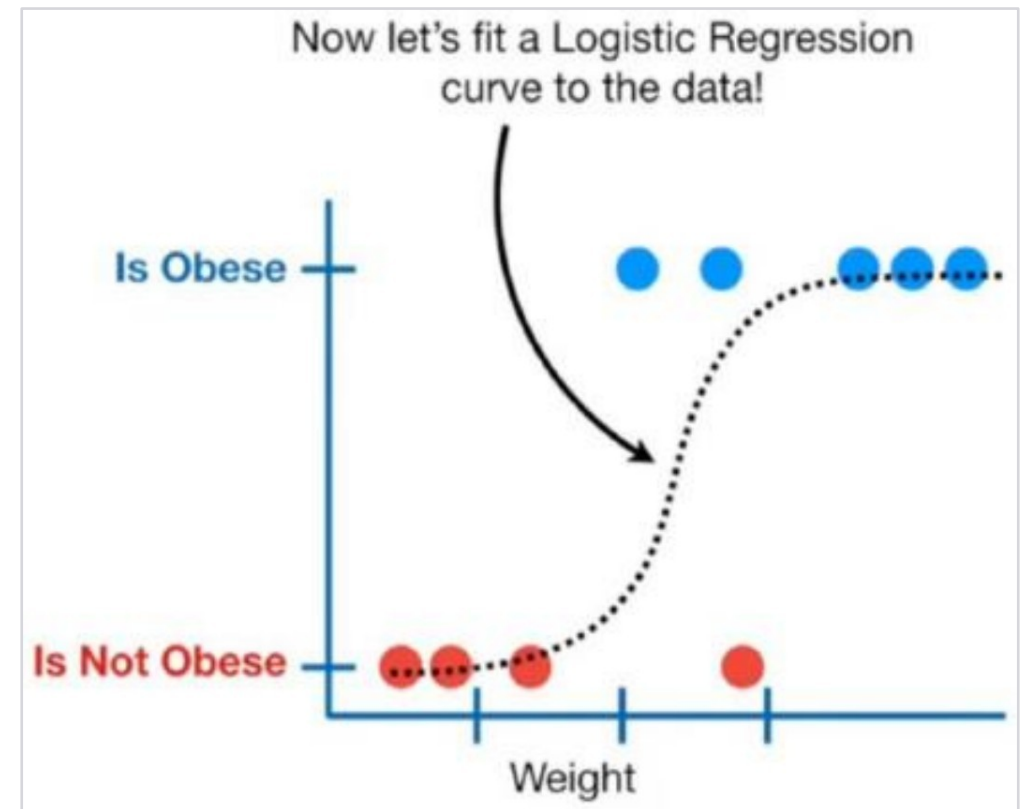
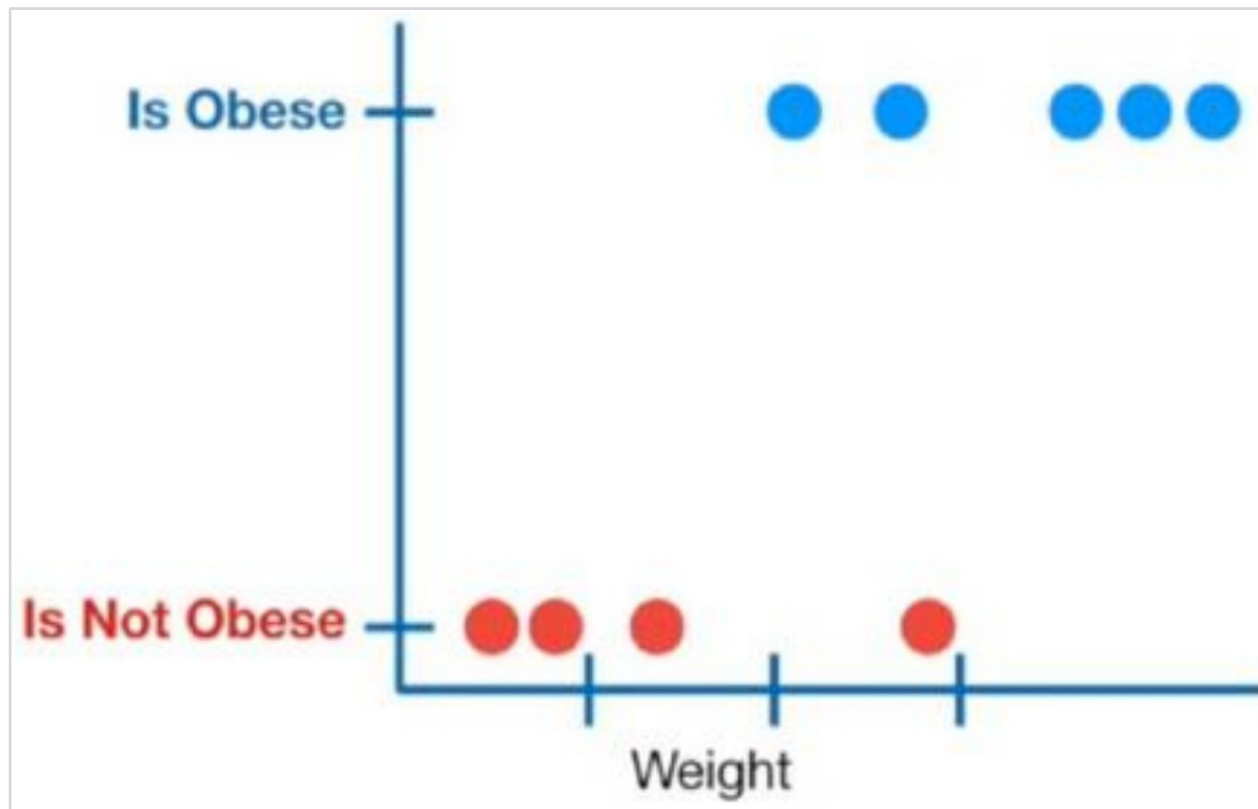
SECTION 02

Basic Setup for Classification Metrics

The most important idea in classification — from probability to decision

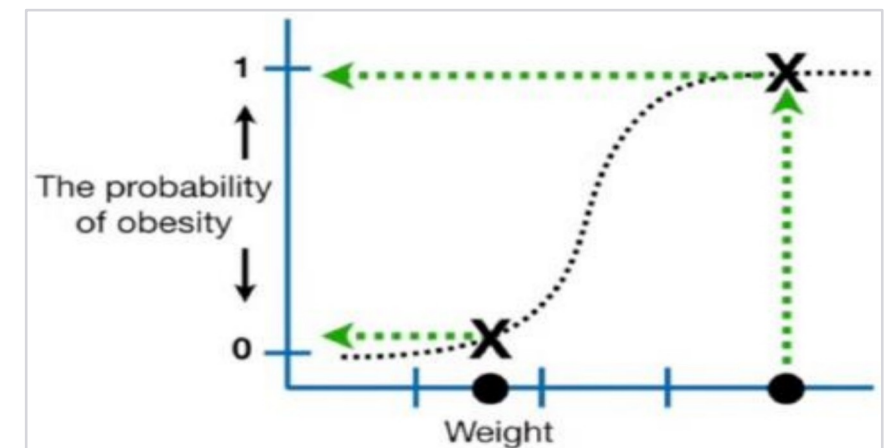
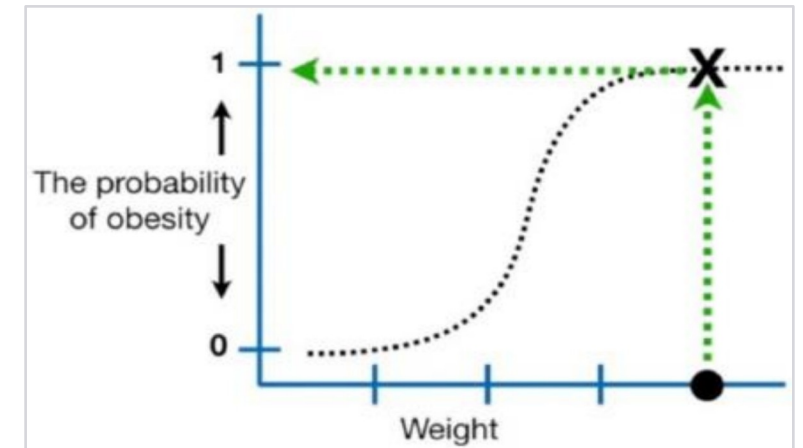
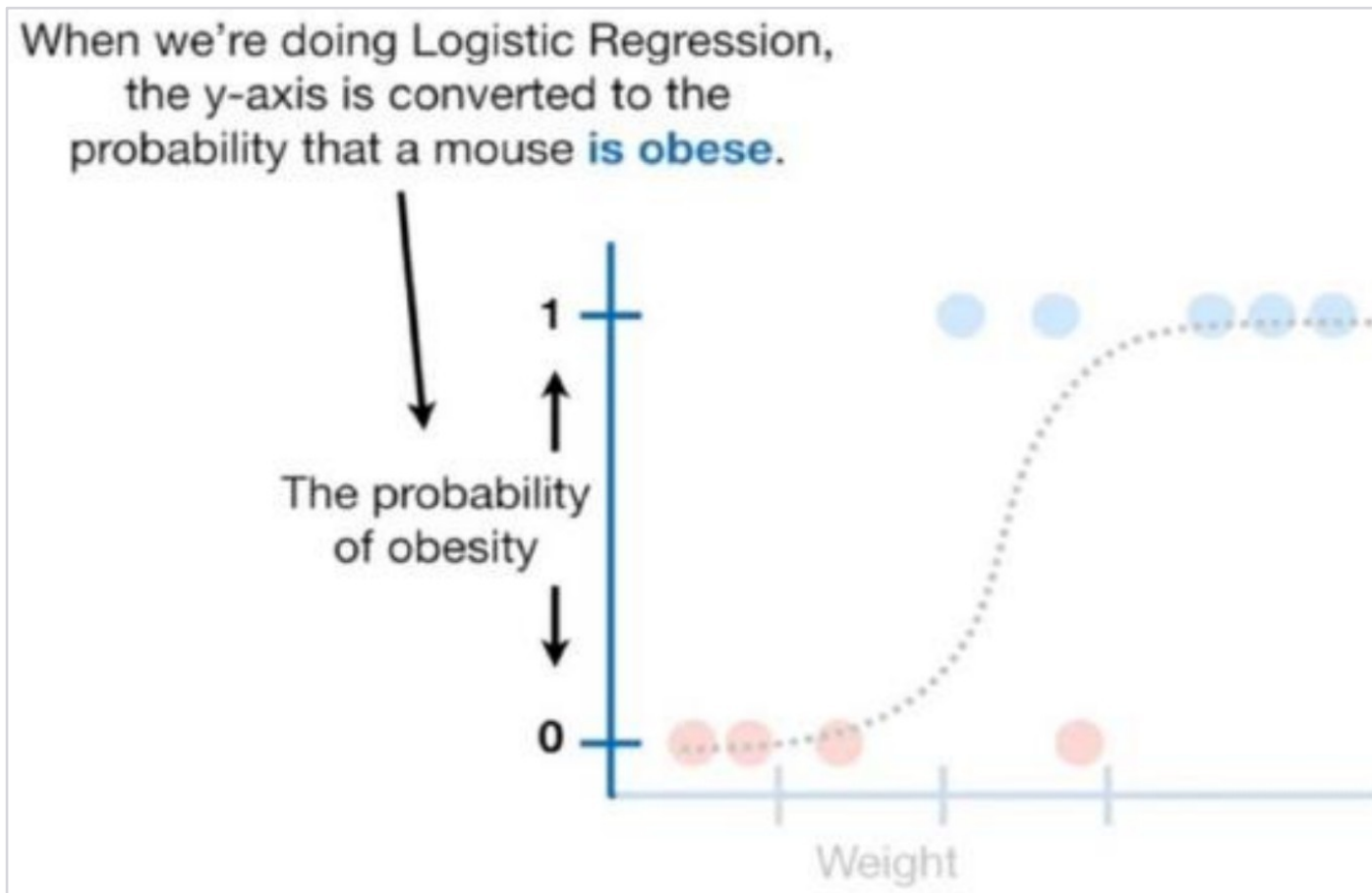
Classifying Obese vs. Not Obese by Weight

Left: the data (blue = obese, red = not obese) · Right: a logistic curve fitted to it



The y-Axis Is the Probability of Obesity

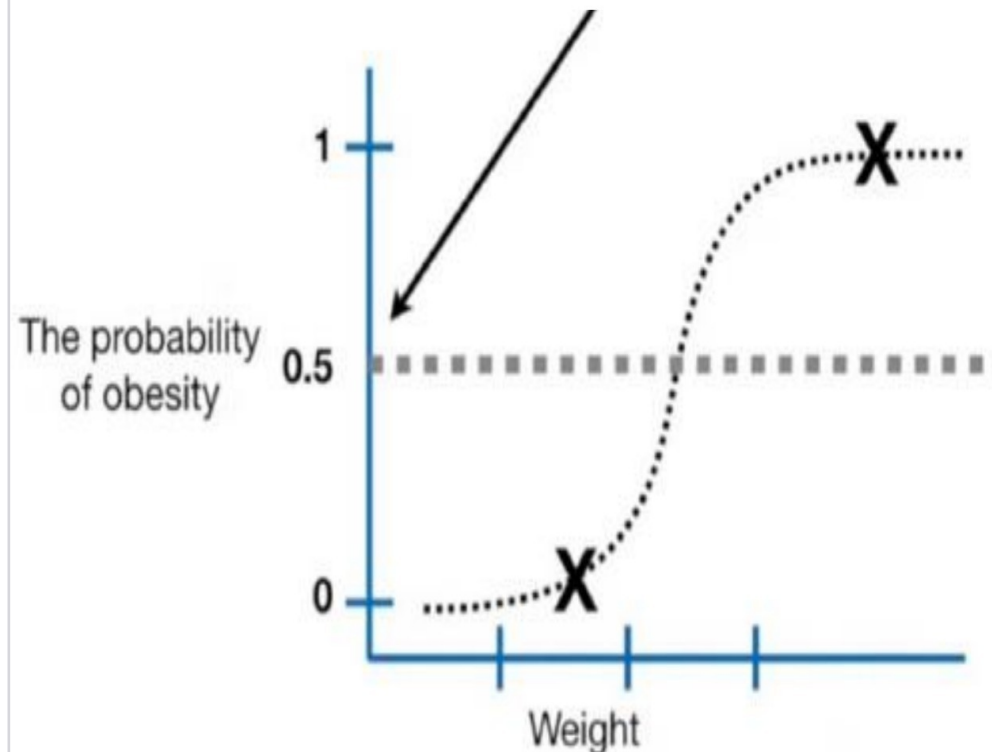
Left: read the y-axis as a probability · Top right: heavy → high probability · Bottom right: light → low



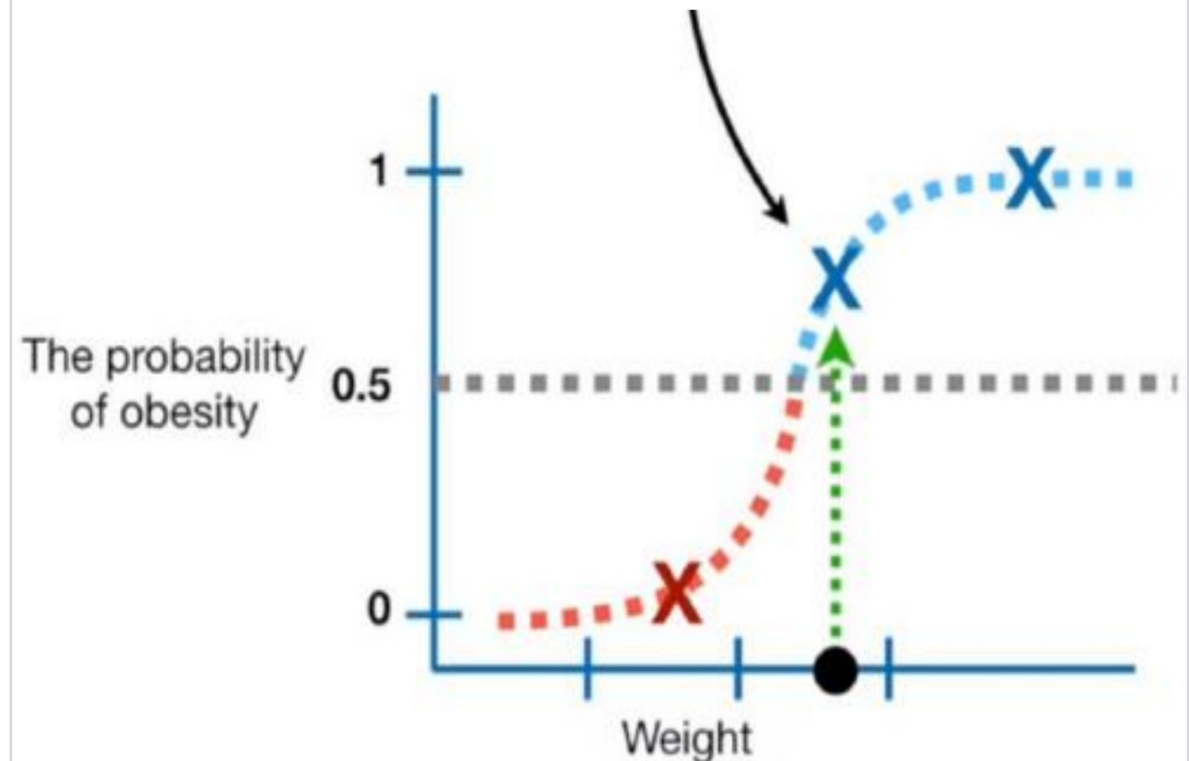
A 0.5 Threshold Turns Probabilities into Calls

Above 0.5 → obese; below 0.5 → not obese

Set the threshold at 0.5 to classify obese vs. not obese.

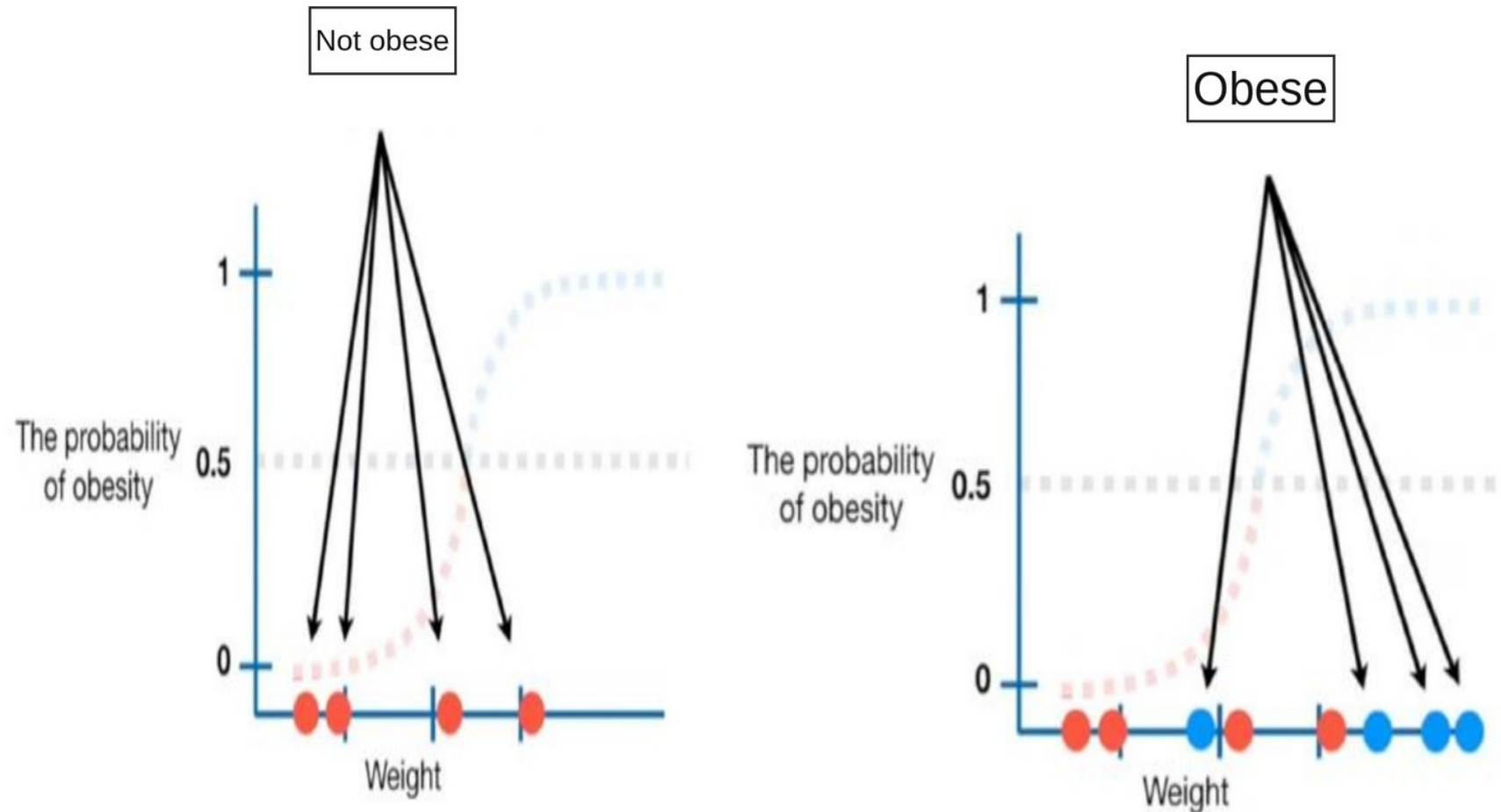


This mouse is classified as obese.



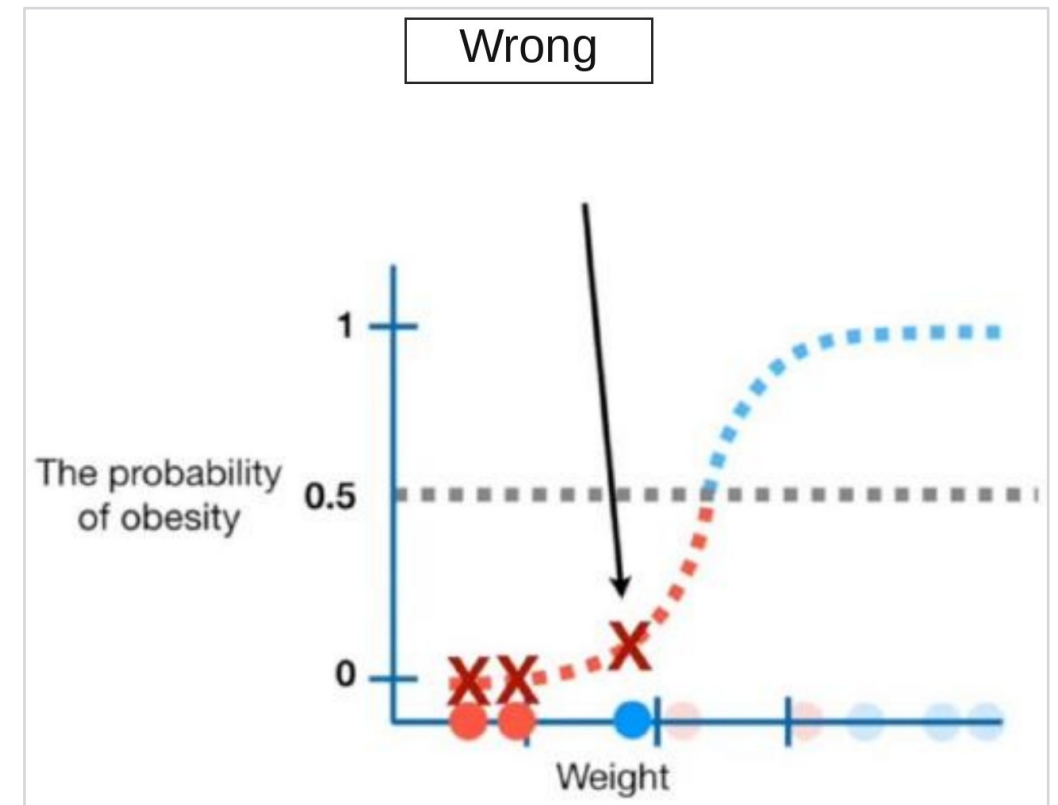
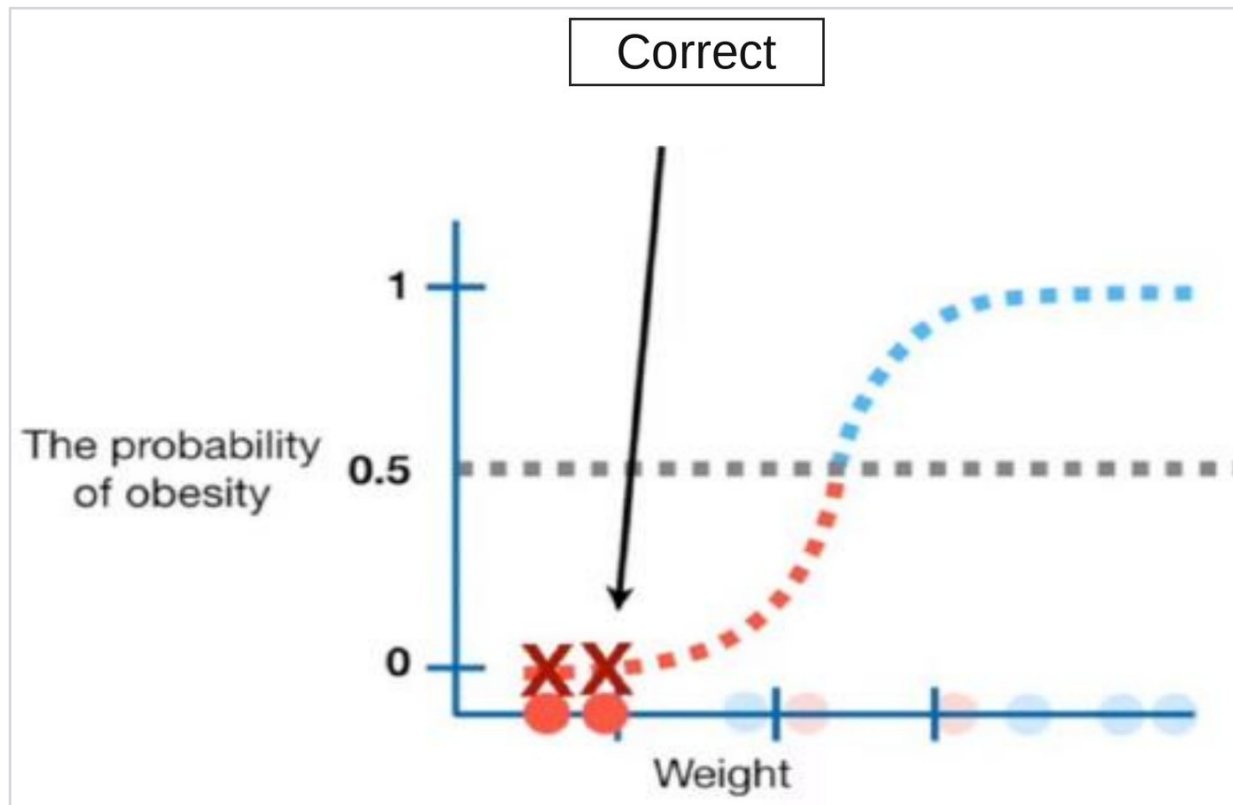
Overlay the True Labels

Red dots are truly not obese, blue dots truly obese — now we can compare calls with the truth



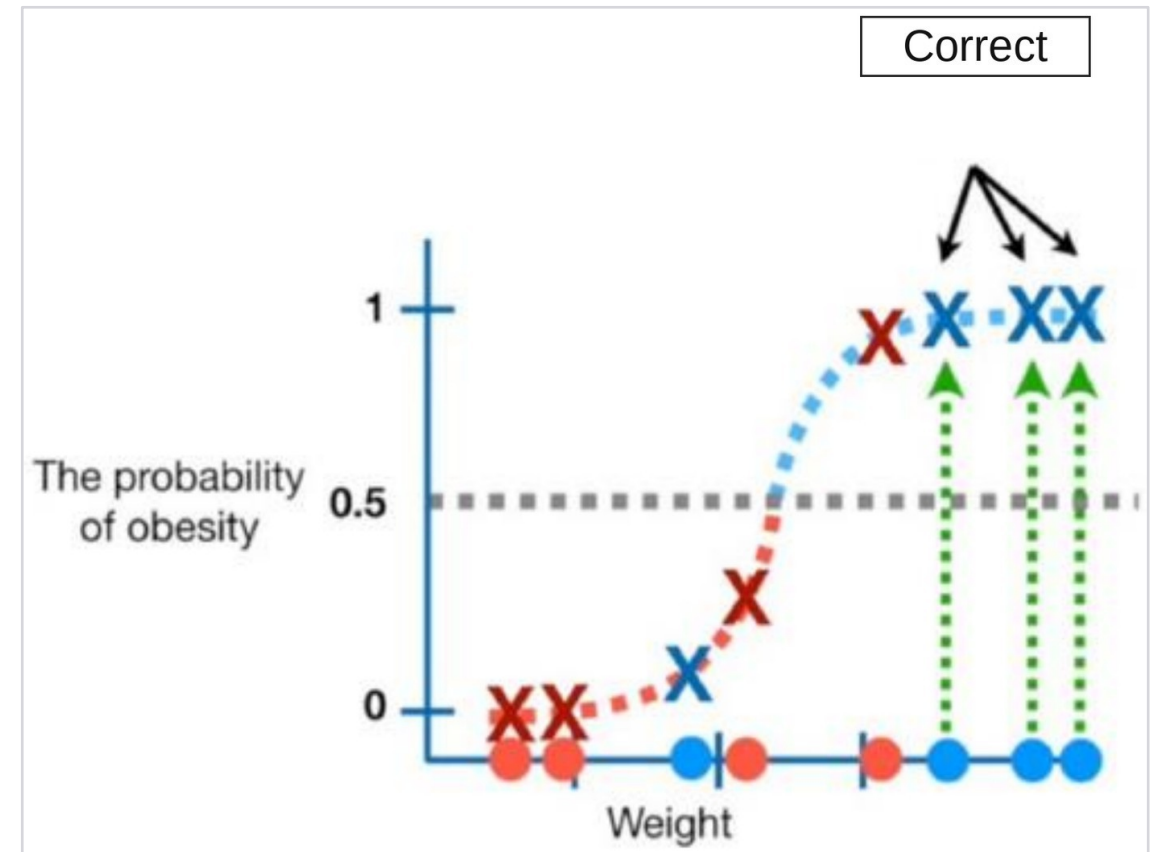
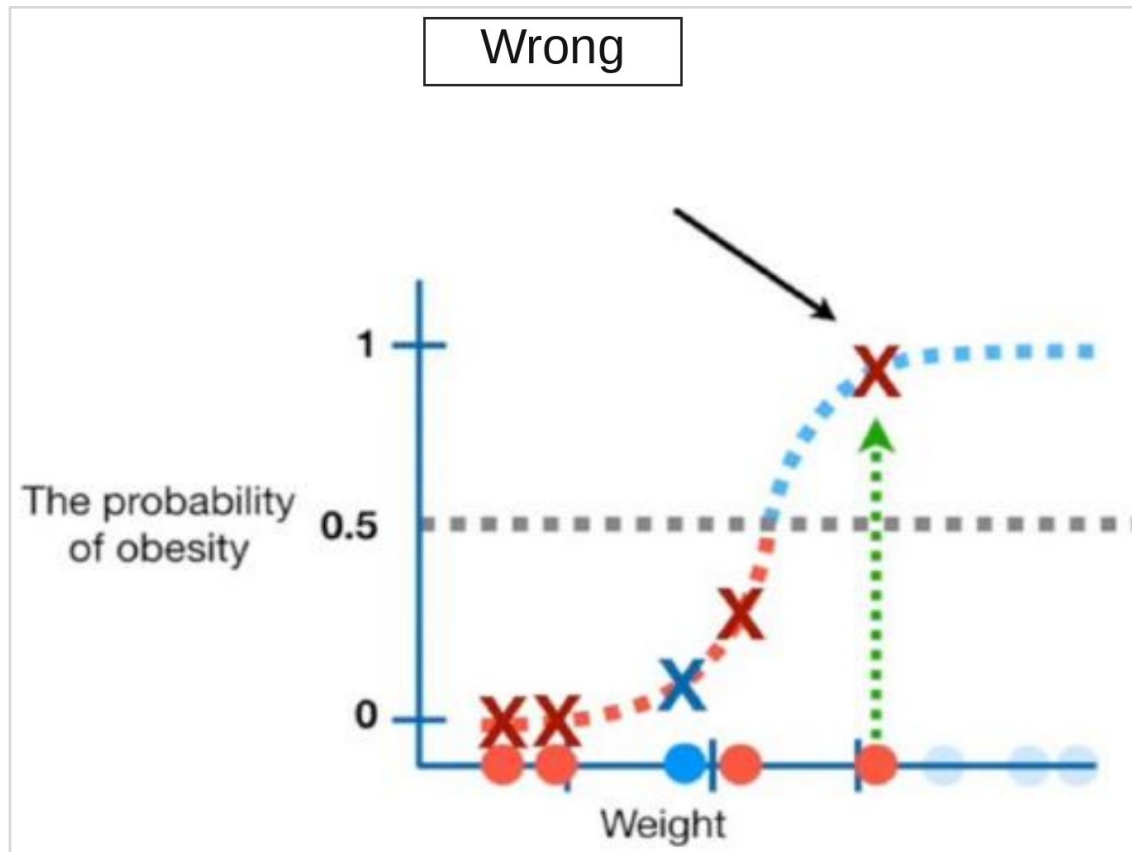
Correct and Wrong Calls (1)

Left: not obese called not obese — correct · Right: obese called not obese — wrong



Correct and Wrong Calls (2)

Left: not obese called obese — wrong · Right: obese called obese — correct. 6 of 8 mice are right



03

SECTION 03

Confusion Matrix and Metrics

Accuracy · recall · precision · $F1$ · $F\beta$ · sensitivity · specificity

The Confusion Matrix

The eight mice in one table: obese 3 · 1, not obese 1 · 3

Mind the layout

- Here: rows = predicted
- sklearn: rows = actual
- Same content, transposed

		Actual	
		Is Obese	Is Not Obese
Predicted	Is Obese	3	1
	Is Not Obese	1	3

The Four Cells and the Basic Metrics

True/False = whether the call was right; Positive/Negative = what the model called

		Actual	
		Has Heart Disease	Does Not Have Heart Disease
Predicted	Has Heart Disease	True Positives	False Positives
	Does Not Have Heart Disease	False Negatives	True Negatives

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall: how many actual positives did we catch? ·

Precision: how many positive calls were right?

Metrics at a Glance

All come from the same four cells — and many go by several names

Metric	Formula	Question it answers	Also called
Accuracy	$(TP+TN) / \text{all}$	How many calls were right?	1 – error rate
Precision	$TP / (TP+FP)$	Of positive calls, how many were positive?	PPV
Recall	$TP / (TP+FN)$	Of actual positives, how many did we catch?	Sensitivity, TPR
Specificity	$TN / (TN+FP)$	Of actual negatives, how many called negative?	TNR
False positive rate	$FP / (FP+TN)$	Of actual negatives, how many called positive?	1 – specificity
F1 score	$2PR / (P+R)$	Are precision and recall both high?	Harmonic mean
$F\beta$ score	$(1+\beta^2)PR / (\beta^2P+R)$	Recall weighted β times as much?	$\beta=2 \rightarrow F2$

Example: Call Everyone a Terrorist?

Ordered to 'catch them at all costs', we flag all 100 people — only one is actually a terrorist

	Actual terrorist	Not terrorist	Total
Predicted terrorist	1 (TP)	99 (FP)	100
Predicted not	0 (FN)	0 (TN)	0
Total	1	99	100

$$P = \frac{1}{1 + 99} = 0.01 \quad R = \frac{1}{1 + 0} = 1 \quad F_1 = \frac{2(0.01)(1)}{0.01 + 1} \approx 0.02$$

The original's 'the average is still 50%' refers to the arithmetic mean — which is why we need the harmonic mean, F1.

Read it as

- Recall 100%, precision 1%
- Accuracy is also 1%
- The arithmetic mean, 50.5%, looks fine
- $F_1 \approx 2\%$ tells the truth
- Next: why recall — and F2 — anyway

F1 Score: Harmonic Mean of Precision and Recall

If either one is near 0, F1 is near 0 — a high F1 needs both to be high

$$F_1 = \frac{2}{\frac{1}{P} + \frac{1}{R}} = \frac{2PR}{P + R}$$

When to use what

- Misses are costly → favor recall
- False alarms are costly → favor precision
- Both matter → F1
- Want to set the weight → F_β

Precision P	Recall R	Arithmetic	F1
0.01	1.00	0.505	0.020
0.50	0.50	0.500	0.500
0.60	0.80	0.700	0.686
0.90	0.10	0.500	0.180

Fixed from the original: 'F!' → F1, 'recal' → recall

When a Miss Is Costly, When a False Alarm Is Costly

If a human reviews every alert, push recall up; if the alert is itself the final action, push precision up

Situation	Cost of a miss (FN)	Cost of a false alarm (FP)	Second gate?	Priority
Airport screening	Mass casualties	A few minutes of search	Yes	Recall · F2
Cancer screening	Treatment delayed	Follow-up test, anxiety	Yes	Recall · F2
Fraud / AML alerts	Loss and penalties	An analyst re-checks	Yes	Recall · F2
Drug-lead triage	A good lead discarded	One more assay	Yes	Recall · F2
Spam filter	One spam seen	A real email is lost	No	Precision · F0.5
Auto-traded signal	Missed opportunity	A real loss booked	No	Precision · F0.5

The terrorist example is a recall case: a second gate makes one FP cheap — yet 1% precision is far too low.

F β Score: How Much Weight to Put on Recall

β = how many times more recall matters than precision; the weight is β^2 , so F2 counts recall 4×

$$F_{\beta} = (1 + \beta^2) \frac{P \cdot R}{\beta^2 P + R}$$

$$F_2 = \frac{5PR}{4P + R} \quad F_{0.5} = \frac{1.25 PR}{0.25P + R}$$

Picking β

- A miss costs $\beta \times$ more $\rightarrow F_{\beta}$
- Misses hurt $\rightarrow F_2$
- False alarms hurt $\rightarrow F_{0.5}$

Metric	Leans toward	Terrorist example
Precision P	—	0.01
Recall R	—	1.00
F0.5	Precision	0.012
F1	Neither	0.020
F2	Recall	0.048

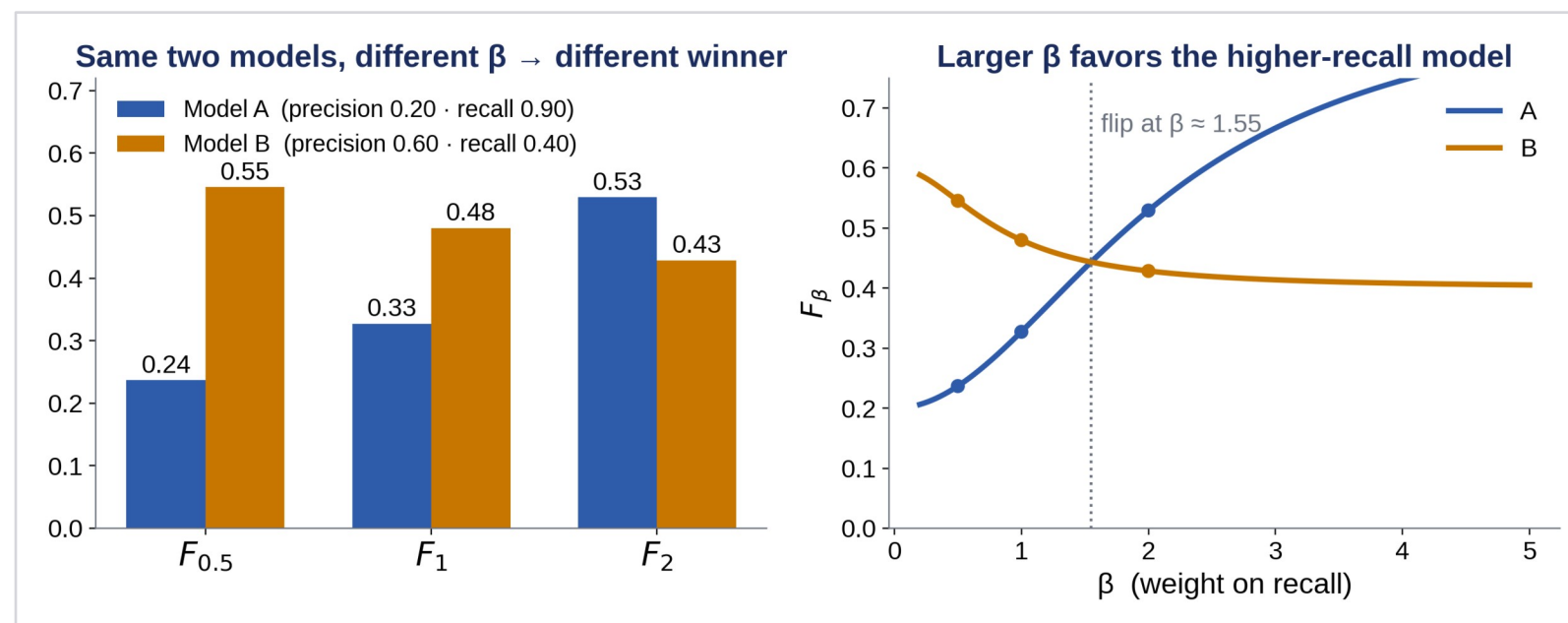
With precision at 0.01 no β rescues the score — F_{β} only sets what you weigh; it never makes a bad model good.

Change β and You Change the Model You Pick

The same two models scored with $F_{0.5}$, F_1 and F_2 — until you fix the metric, 'the better model' is undefined

What to see

- A: P 0.20 / R 0.90
- B: P 0.60 / R 0.40
- F_1 : B wins
- F_2 : A wins
- flip at $\beta \approx 1.55$



Sensitivity and Specificity

Sensitivity (= recall): positives called positive · Specificity: negatives called negative

$$\text{Sensitivity} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$

Actual

	Has Heart Disease	Does Not Have Heart Disease								
Predicted	<table><tr><td>Has Heart Disease</td><td>True Positives</td></tr><tr><td>Does Not Have Heart Disease</td><td>False Negatives</td></tr></table>	Has Heart Disease	True Positives	Does Not Have Heart Disease	False Negatives	<table><tr><td>Has Heart Disease</td><td>False Positives</td></tr><tr><td>Does Not Have Heart Disease</td><td>True Negatives</td></tr></table>	Has Heart Disease	False Positives	Does Not Have Heart Disease	True Negatives
Has Heart Disease	True Positives									
Does Not Have Heart Disease	False Negatives									
Has Heart Disease	False Positives									
Does Not Have Heart Disease	True Negatives									

Specificity = $\frac{\text{True Negatives}}{\text{True Negatives} + \text{False Positives}}$

Actual

		Has Heart Disease	Does Not Have Heart Disease
Predicted	Has Heart Disease	True Positives	False Positives
	Does Not Have Heart Disease	False Negatives	True Negatives

Computing Sensitivity and Specificity

Heart-disease example: 303 patients, 171 with heart disease and 132 without

	Has Heart Disease	Does Not Have Heart Disease
Has Heart Disease	139	20
Does Not Have Heart Disease	32	112

$$\text{Sensitivity} = \frac{139}{139 + 32} = 0.81 \quad \text{Specificity} = \frac{112}{112 + 20} = 0.85$$

Metric	Calculation	Value
Precision	$139 / (139 + 20)$	0.874
Accuracy	$(139 + 112) / 303$	0.828
F1	$2PR / (P + R)$	0.842

The same model can look different depending on the metric — decide which metric fits the purpose first.

04

SECTION 04

ROC Curve

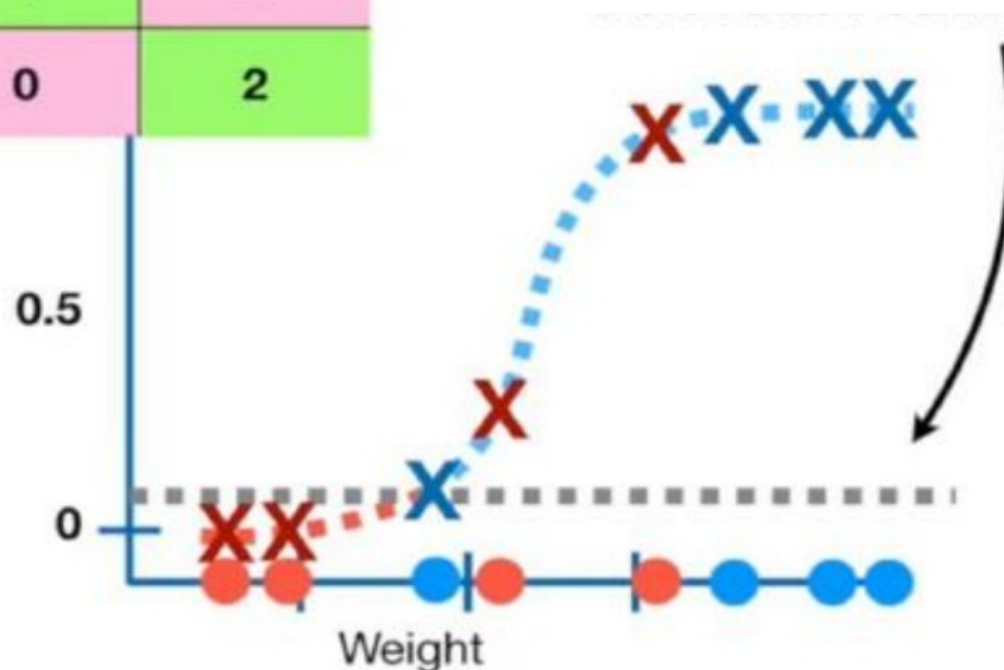
Moving the threshold changes the confusion matrix — see every case on one chart

Lowering the Threshold to 0.1

Every obese mouse is caught (TP 4, FN 0), but two not-obese mice are wrongly called obese (FP 2)

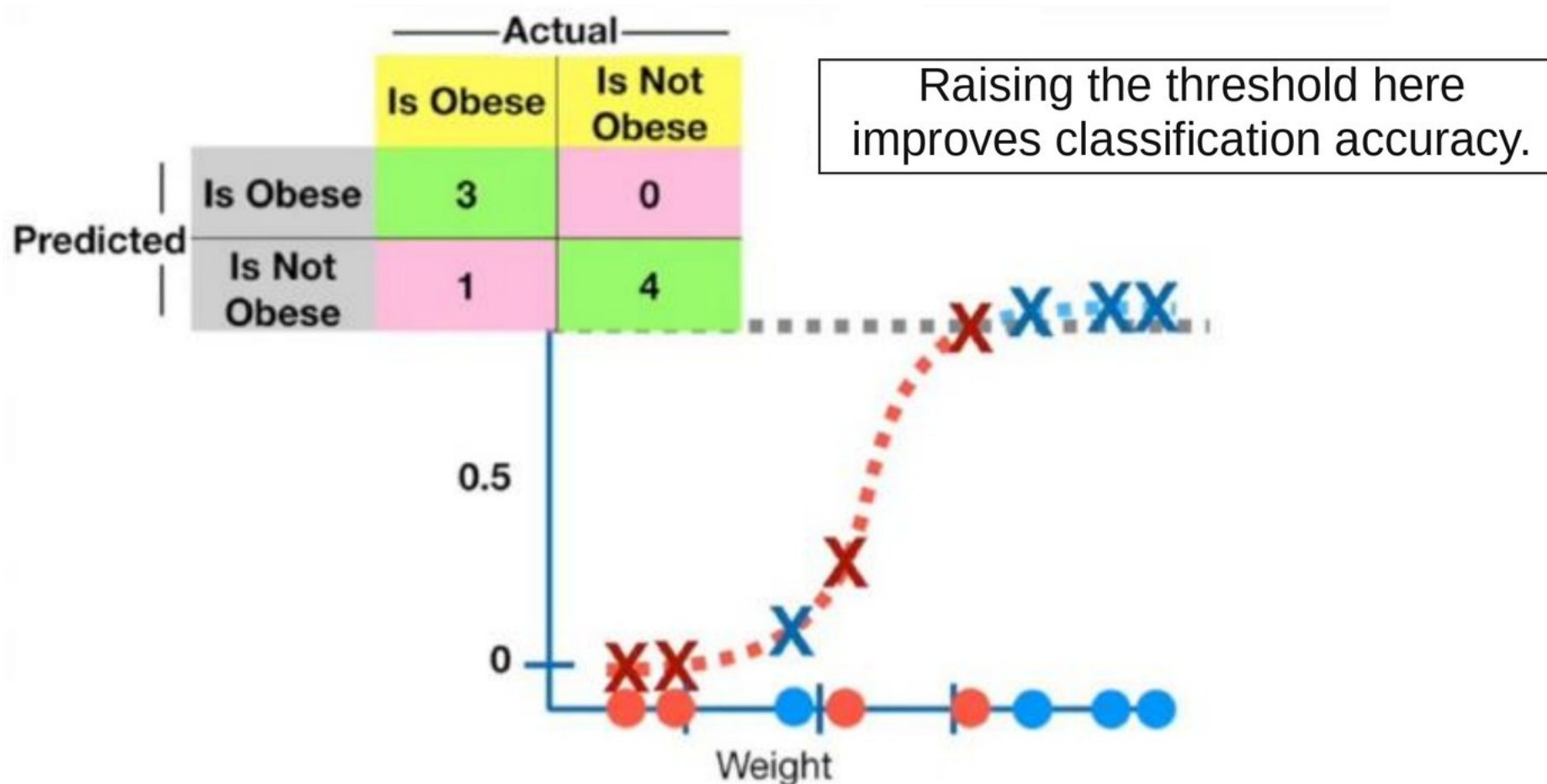
		Actual	
		Infected	Not Infected
Predicted	Infected	4	2
	Not Infected	0	2

Lower threshold: more false positives (predicted obese but wrong), but every obese mouse is caught.



Raising the Threshold to 0.9

Every not-obese mouse is right (FP 0), one obese mouse is missed (FN 1): accuracy $7/8 = 0.875$

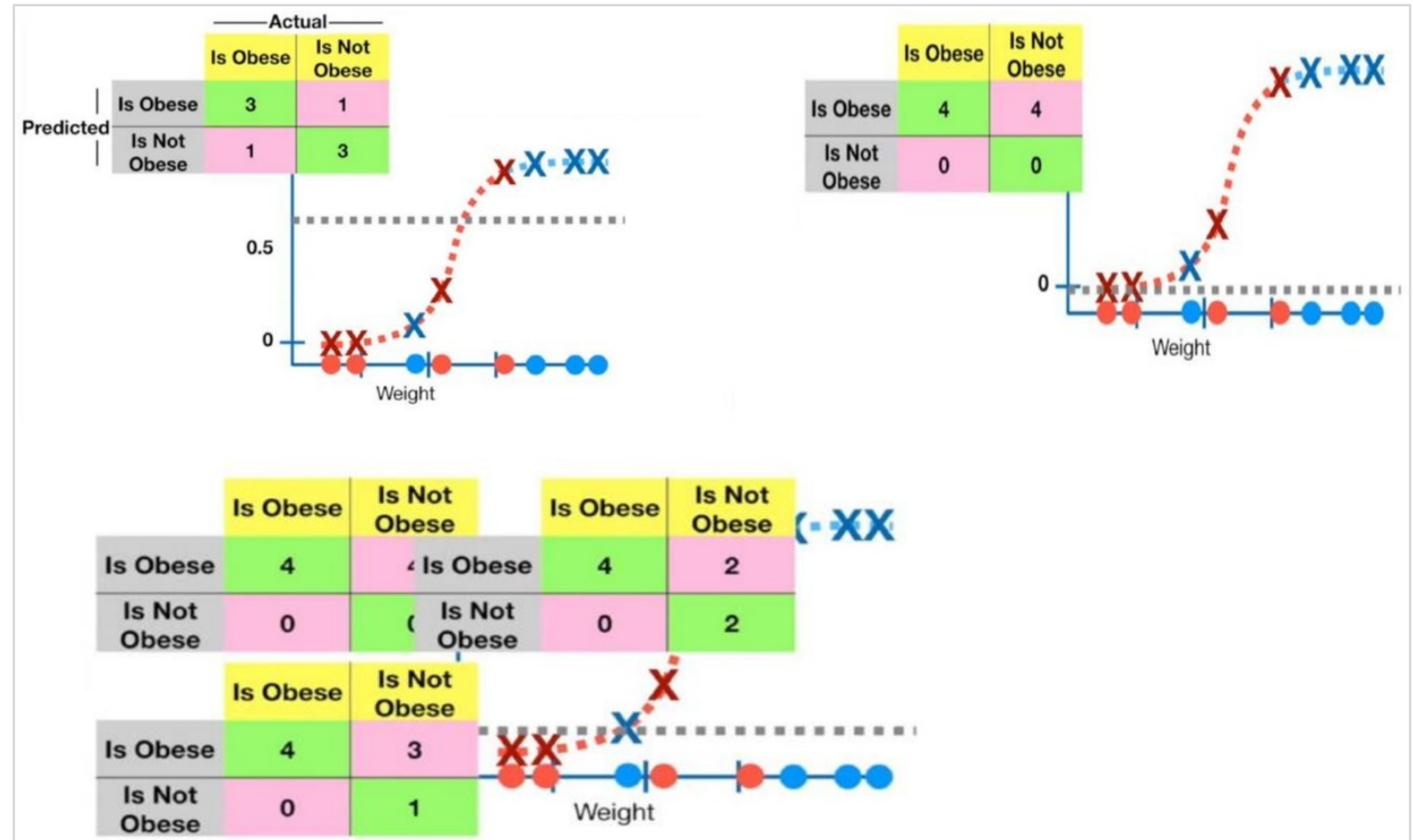


How Do We Choose the Threshold?

A confusion matrix for every threshold means far too many tables to compare

The problem

- Countless thresholds
- Many tables are hard to compare
- → one chart instead

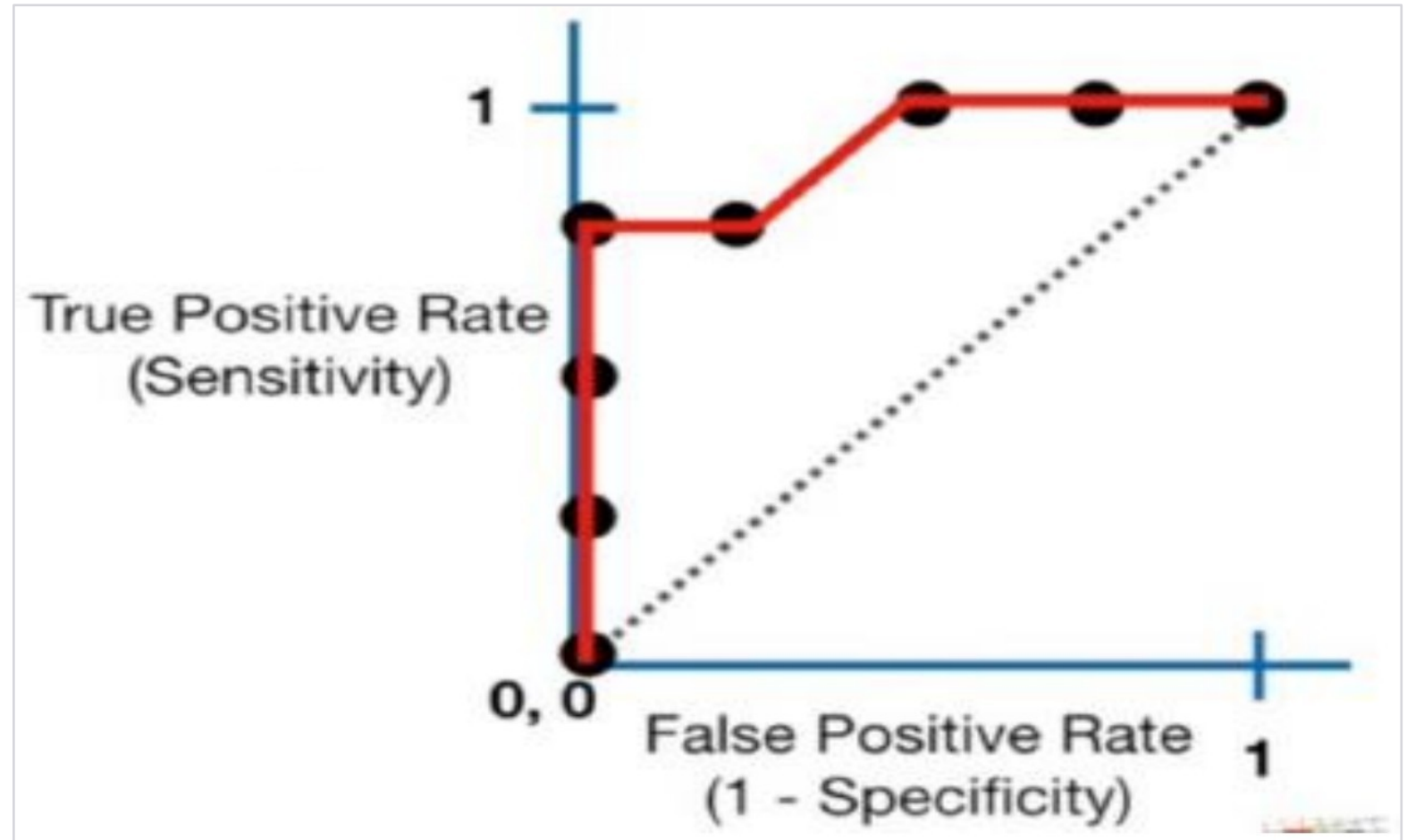


The ROC Graph

ROC (Receiver Operating Characteristic): one (false positive rate, true positive rate) point per threshold

Anatomy

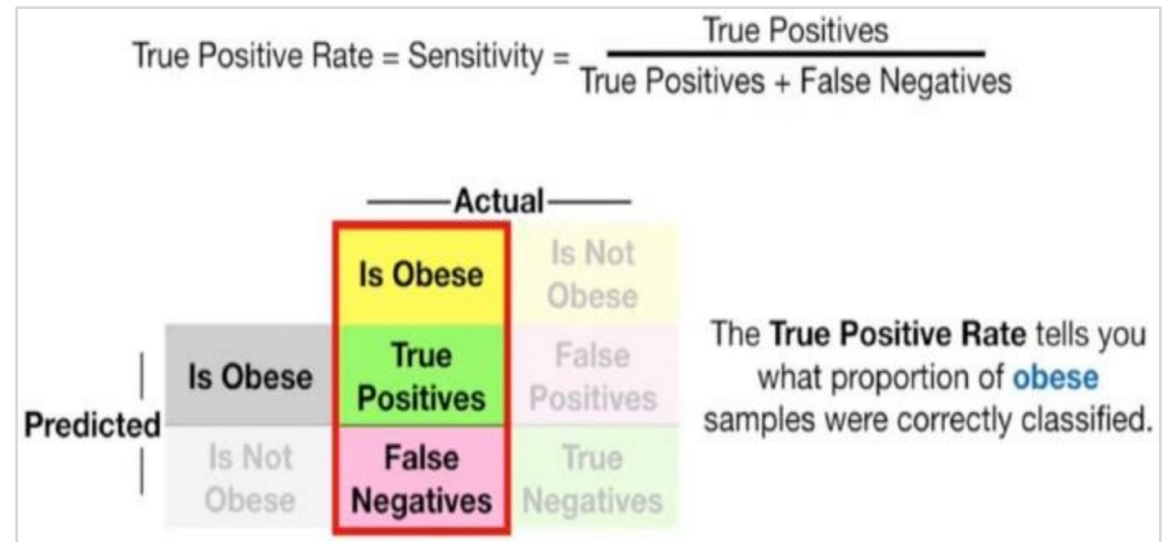
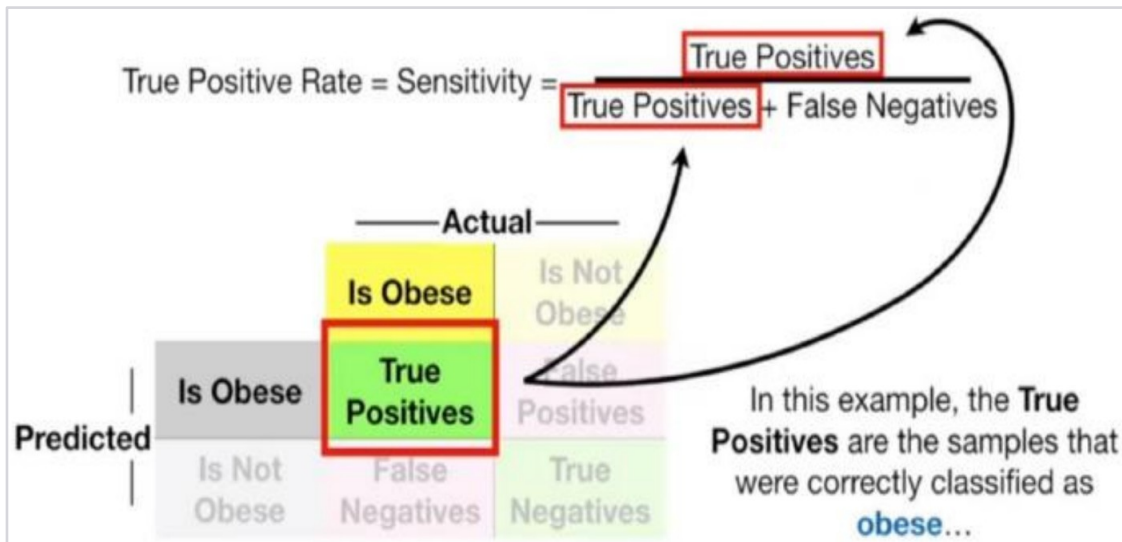
- y: true positive rate = sensitivity
- x: false positive rate = $1 - \text{specificity}$
- One point = one threshold



y-Axis: True Positive Rate (Sensitivity)

The share of truly obese mice that were called obese

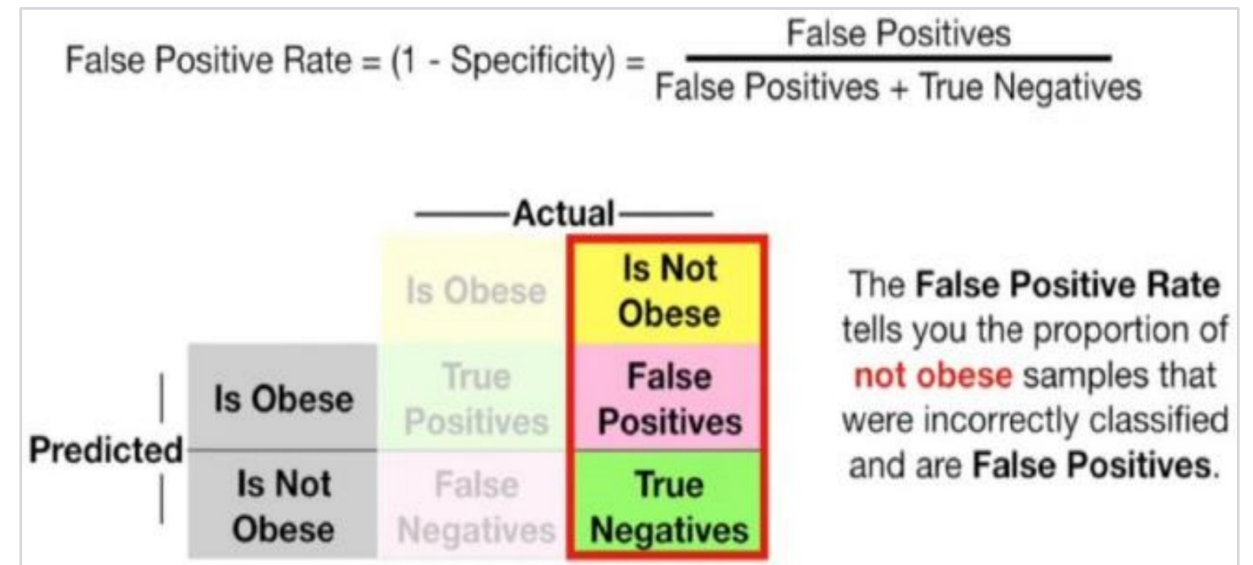
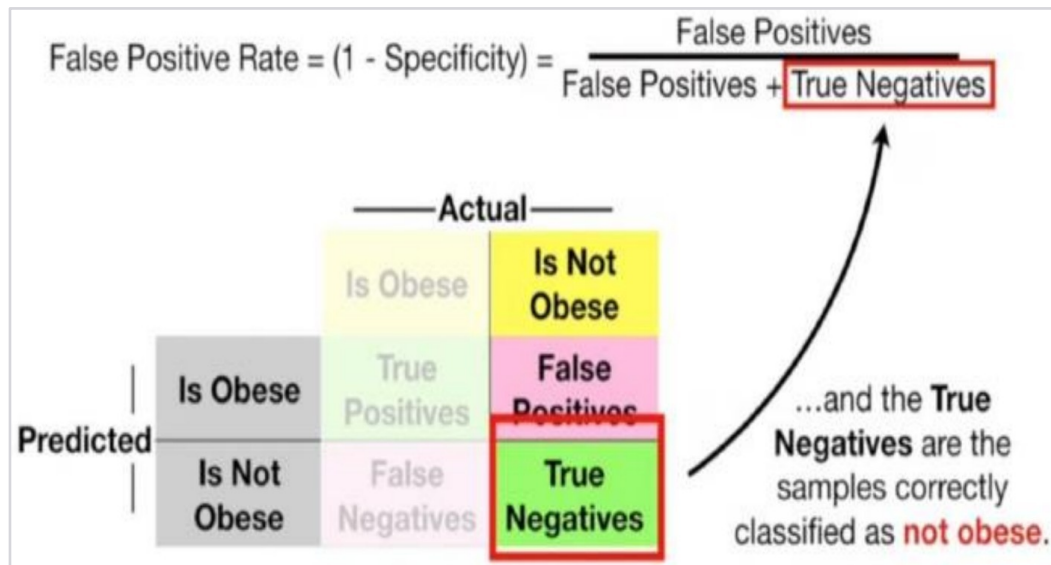
$$\text{TPR} = \text{sensitivity} = \frac{TP}{TP + FN}$$



x-Axis: False Positive Rate (1 – Specificity)

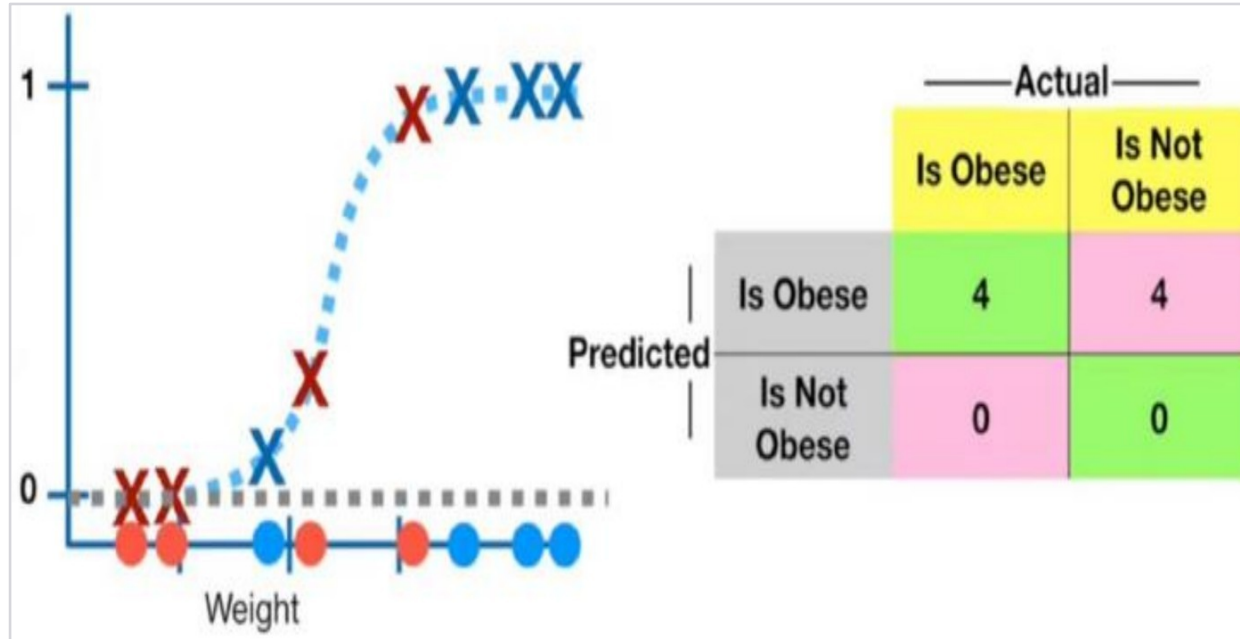
The share of truly not-obese mice that were wrongly called obese

$$\text{FPR} = 1 - \text{specificity} = \frac{FP}{FP + TN}$$

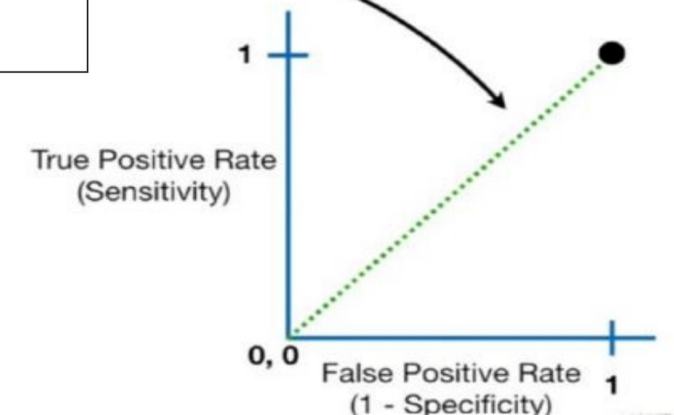


Very Low Threshold: Point (1, 1)

Everything called obese → TPR = 1, FPR = 1. The green diagonal (TPR = FPR) is guessing level

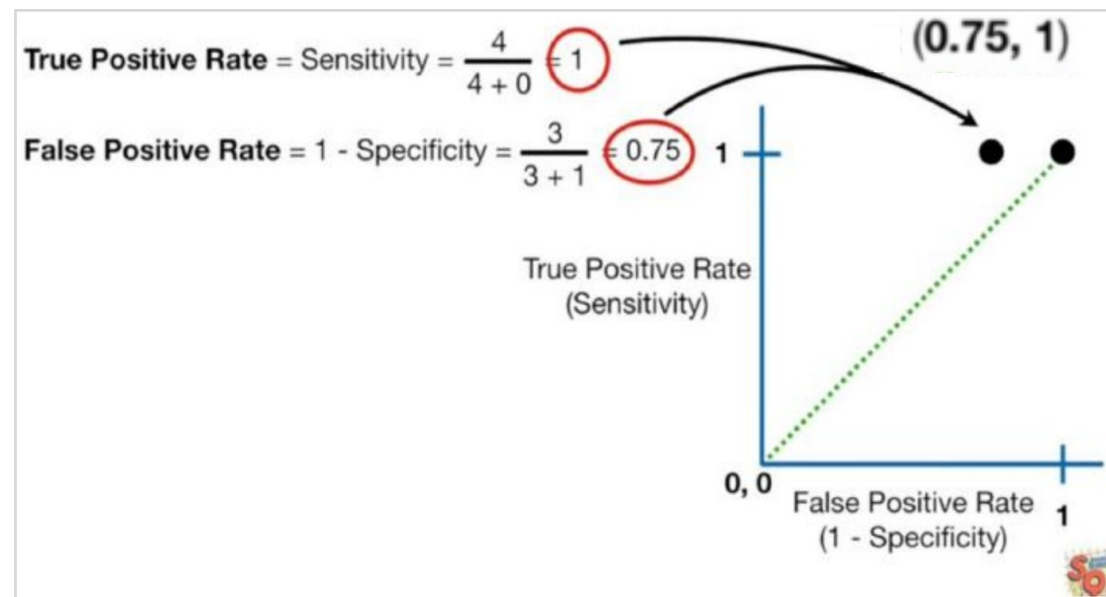
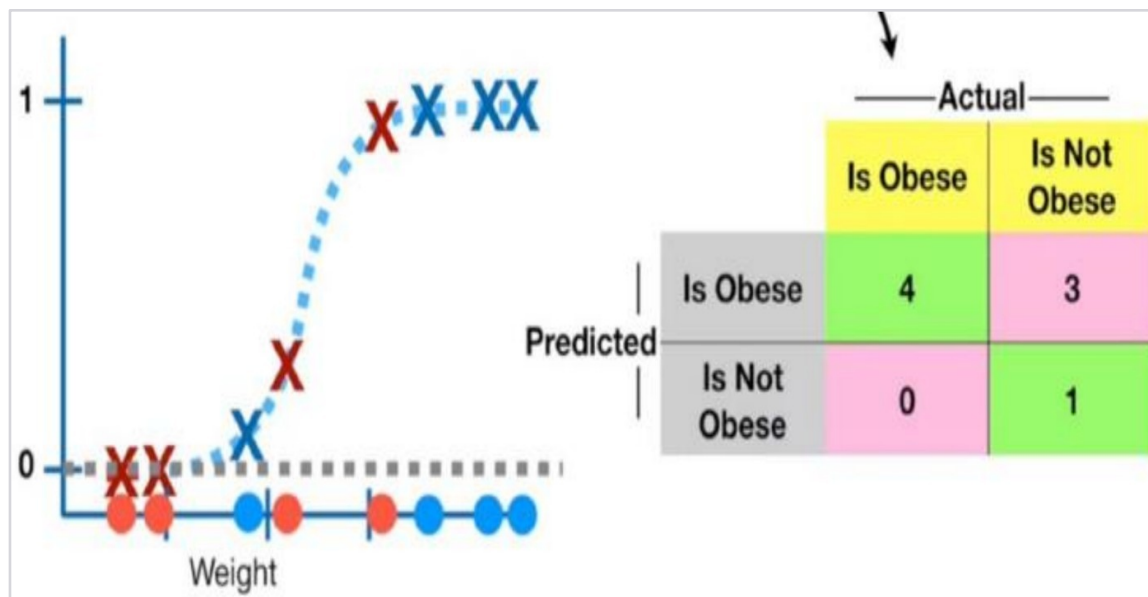


The green line marks TPR = FPR: obese mice are classified correctly at the same rate as not-obese mice are misclassified as obese.



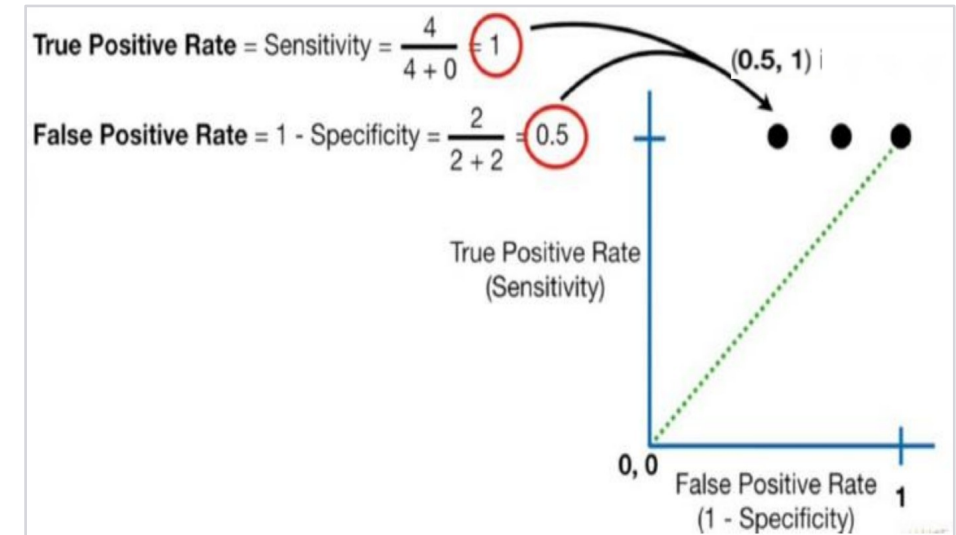
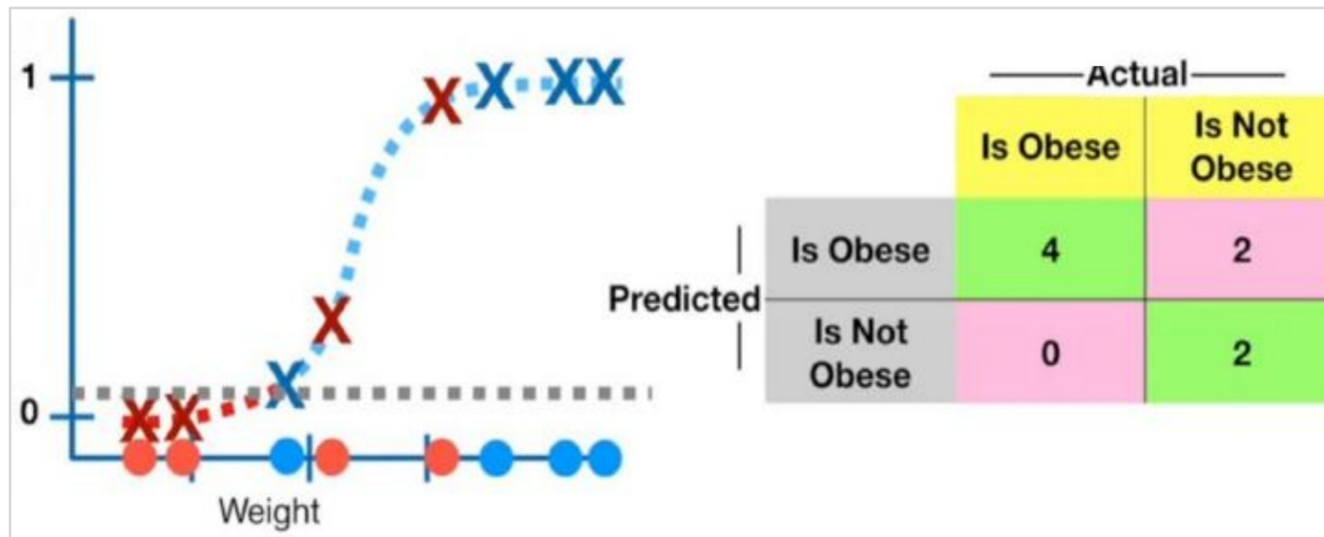
A Little Higher: Point (0.75, 1)

One not-obese mouse called correctly → TPR 1, FPR 0.75: left of the diagonal, better than before



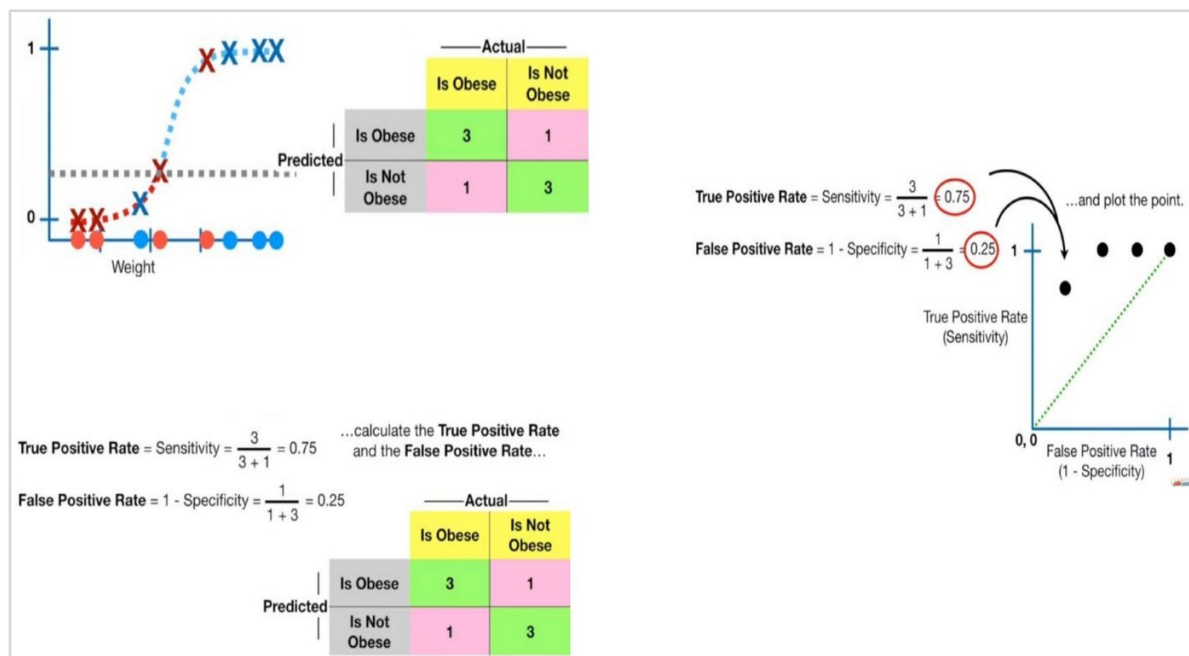
Higher Again: Point (0.5, 1)

TPR stays at 1 while FPR falls to $2/4 = 0.5$

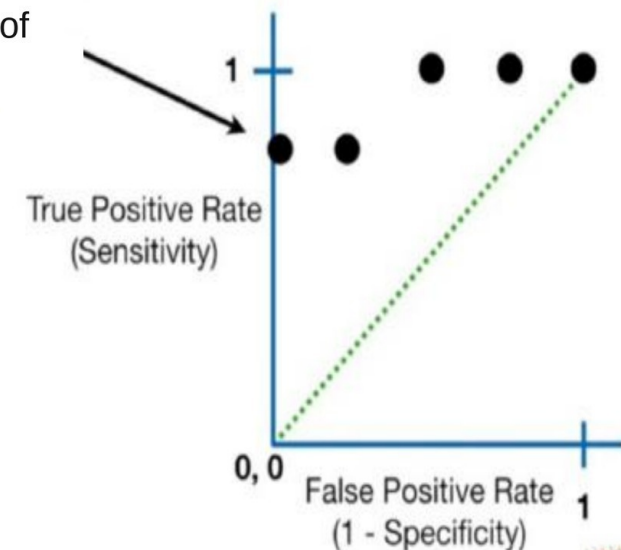


Keep Raising: (0.25, 0.75) → (0, 0.75)

The threshold at (0, 0.75) catches 75% of obese mice while getting 100% of not-obese mice right

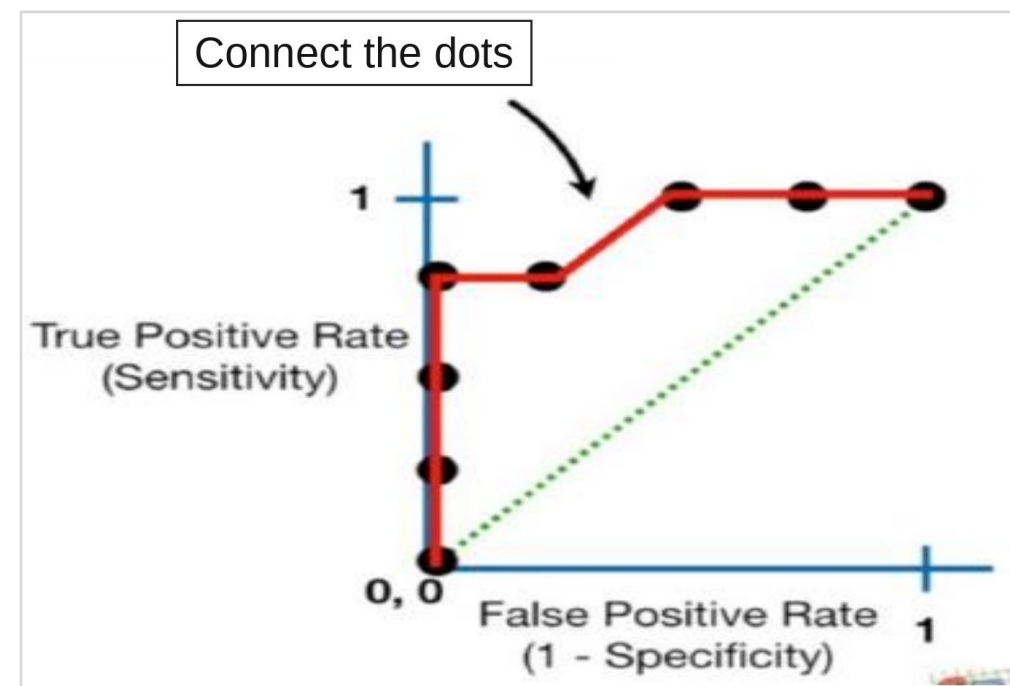
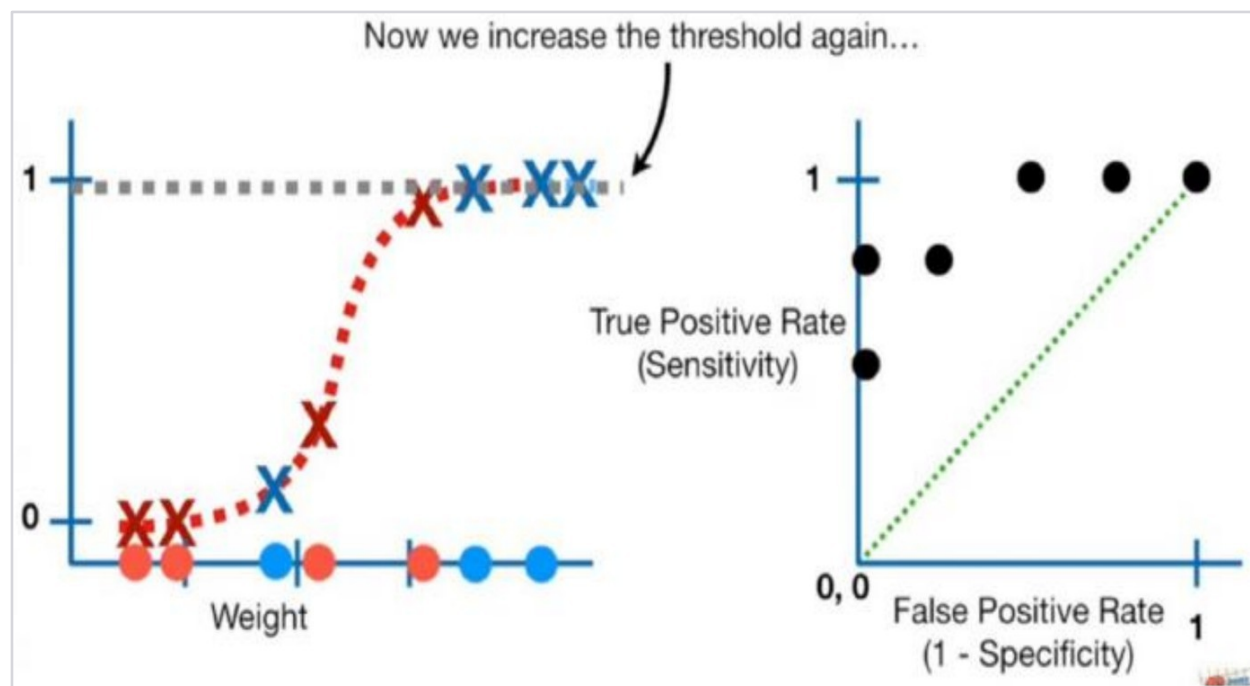


The new point (0, 0.75): this threshold correctly classifies 75% of obese and 100% of not-obese samples.



Raise It All the Way, Then Connect the Dots

The highest threshold gives (0, 0) — joining all the points gives the ROC curve



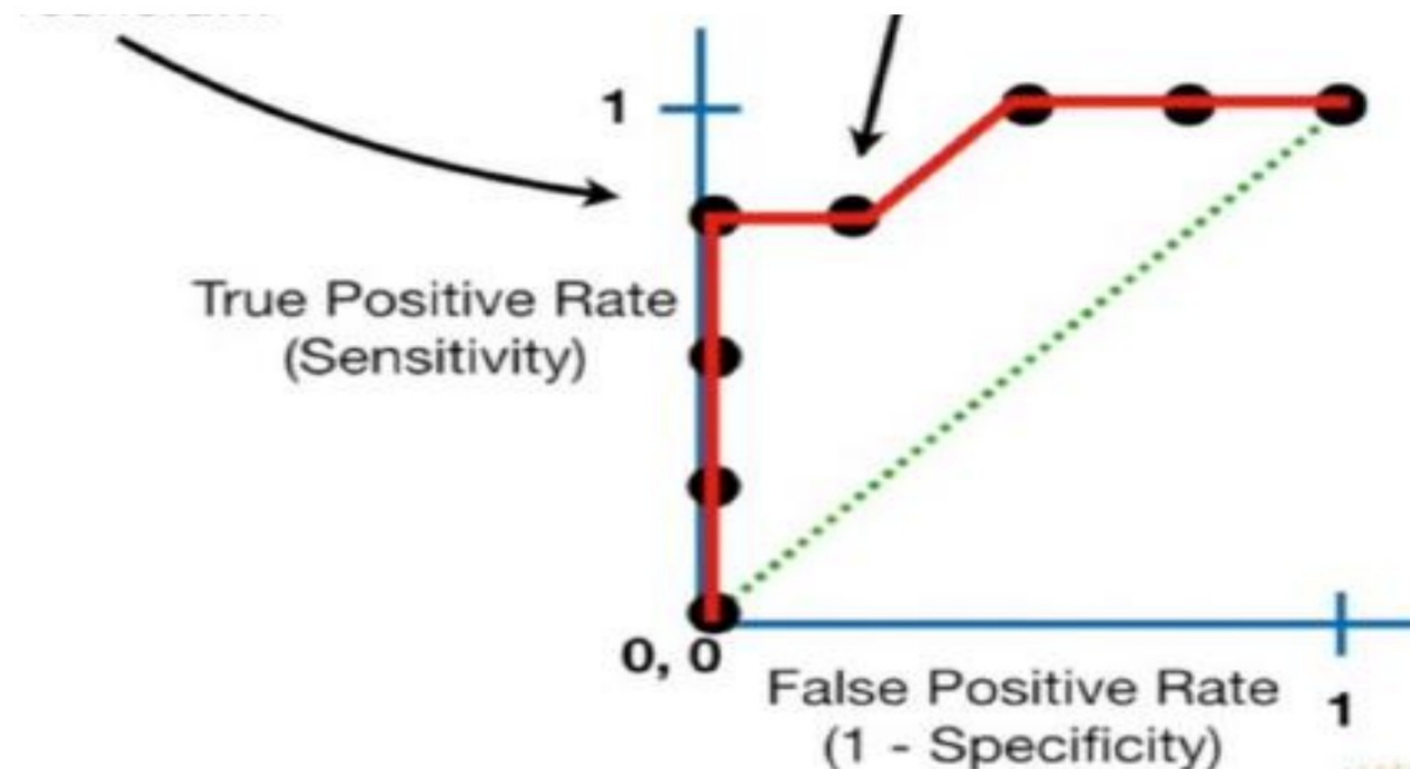
Toward the Top Left Is Better

At equal TPR, lower FPR wins: $(0, 0.75)$ beats $(0.25, 0.75)$

Read it as

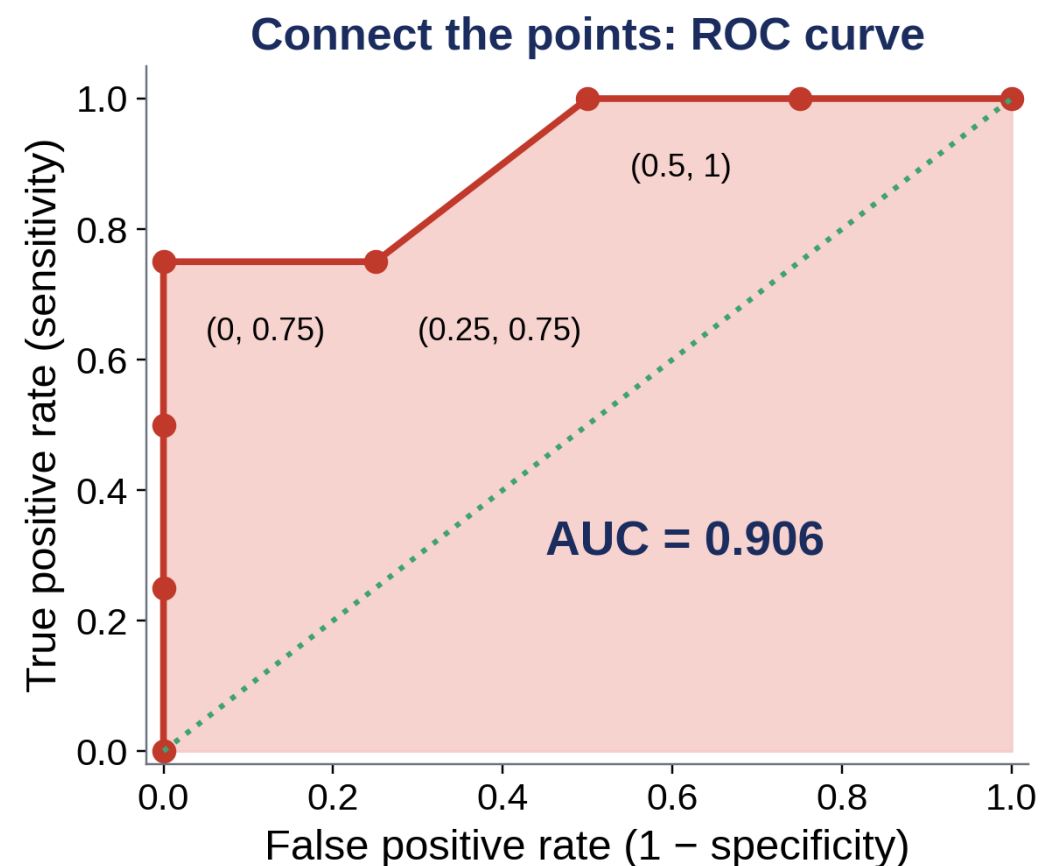
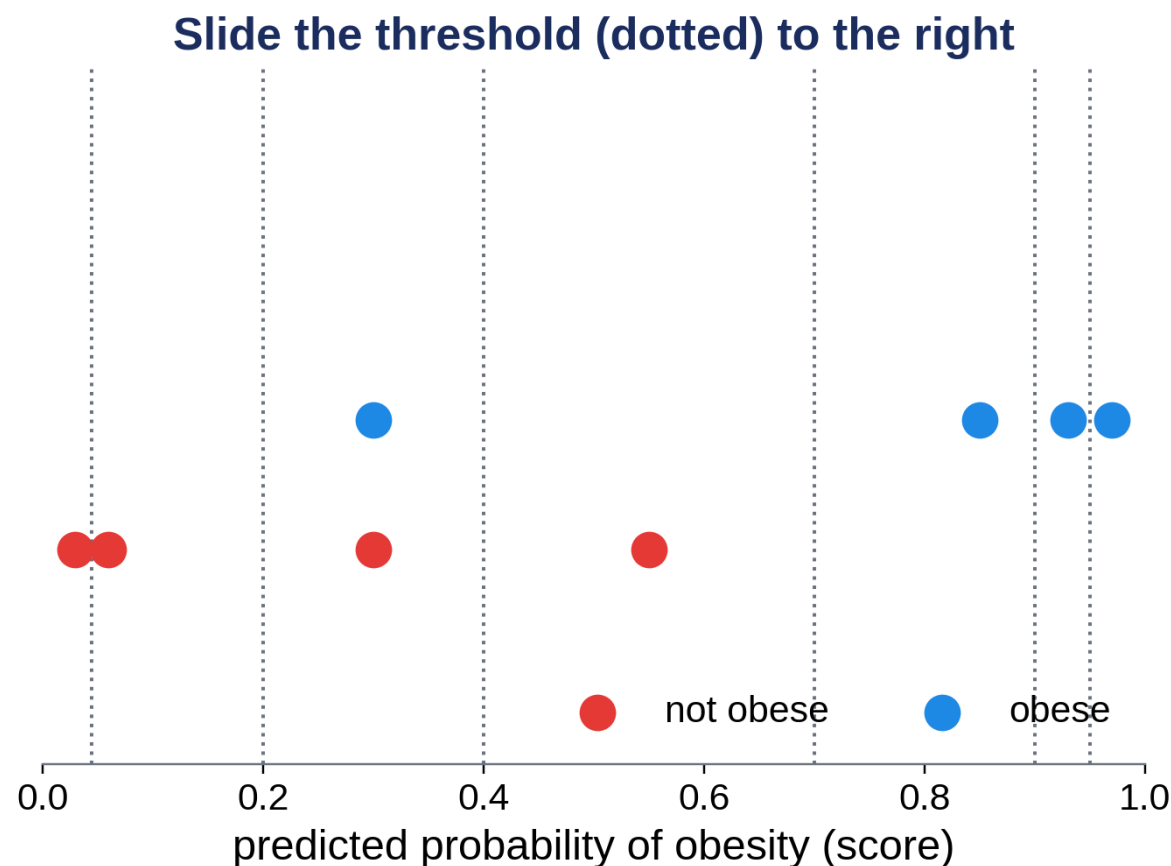
- Top left $(0, 1)$ = perfect
- Diagonal = random
- Below diagonal = inverted

The threshold on the left is better than this one.



Building an ROC Curve on One Page

Sort the scores, step the threshold, plot (FPR, TPR) — the same example, recomputed



Choose the Threshold by the Cost of Errors

The ROC curve shows the options; the business decides which point to use

$$J = \text{TPR} - \text{FPR} = \text{sensitivity} + \text{specificity} - 1$$

When both errors cost about the same: pick the threshold with the largest Youden index J

Costly misses → lower it

- Cancer screening, fraud detection
- Favor recall (sensitivity)
- Filter false alarms with a second check

Costly false alarms → raise it

- Spam filters, trading signals (costs)
- Favor precision (specificity)
- Act only when confident

05

SECTION 05

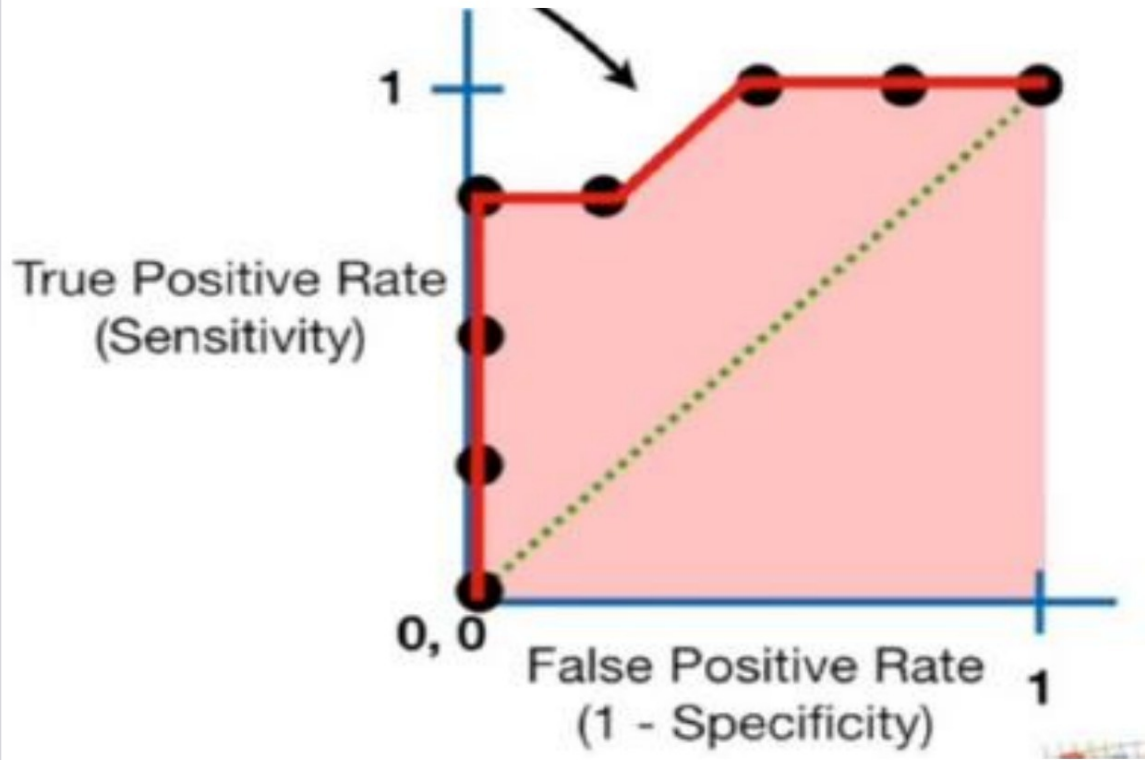
AUC and Precision–Recall Curves

A whole model in one number — and what to watch with imbalanced data

ROC AUC: Area Under the Curve

AUC = 0.9 in the example (0.906 exactly) — it measures ranking ability, independent of the threshold

AUC (area under the curve) = 0.9



$$\text{AUC} = P(\hat{s}(x^+) > \hat{s}(x^-))$$

Read it as

- 1 = perfect, 0.5 = random
- Probability a random positive scores higher than a random negative
- Example: 14.5 of 16 pairs $\rightarrow$ 0.906

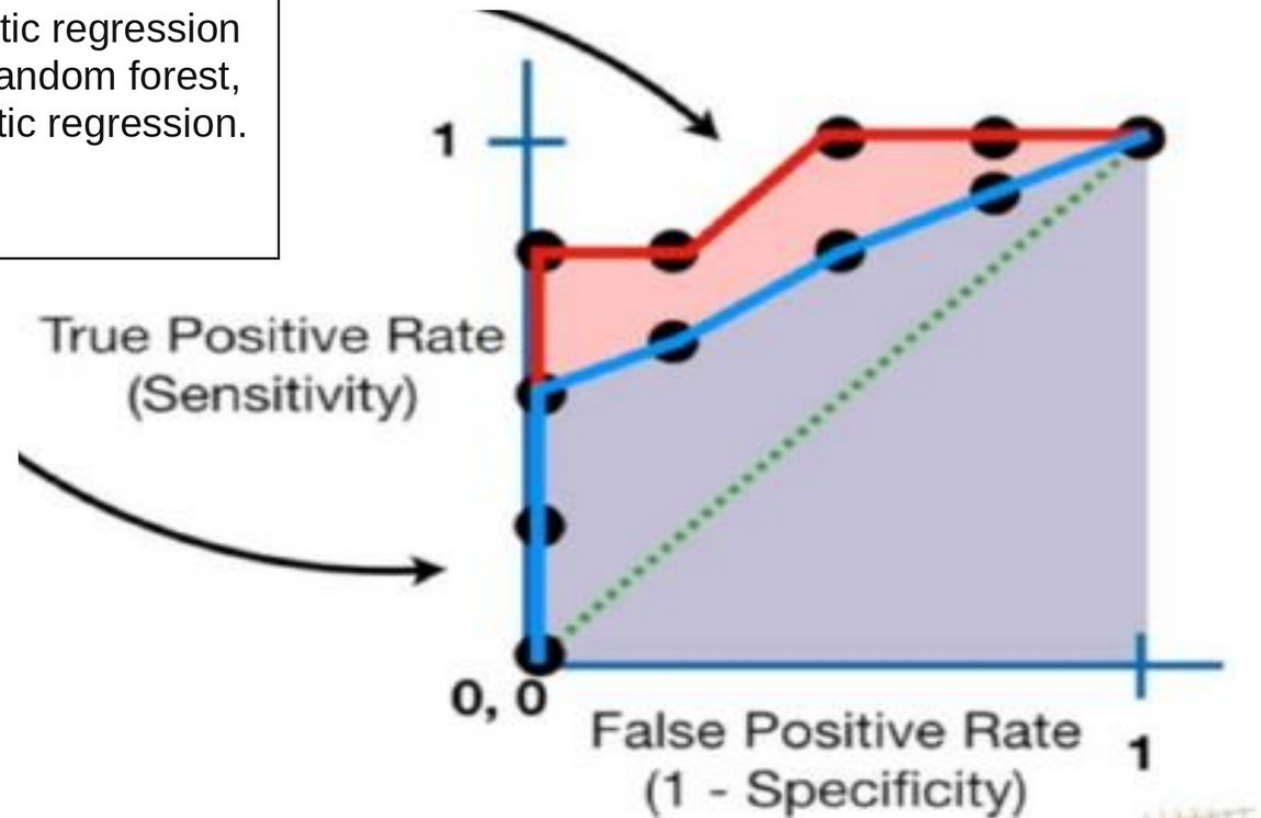
Comparing Models with AUC

Red (logistic regression) has more area than blue (random forest) → choose logistic regression

Caution

- If curves cross, AUC is not enough
- Look at the FPR range you will use
- AUC ignores calibration

If red = logistic regression
and blue = random forest,
choose logistic regression.

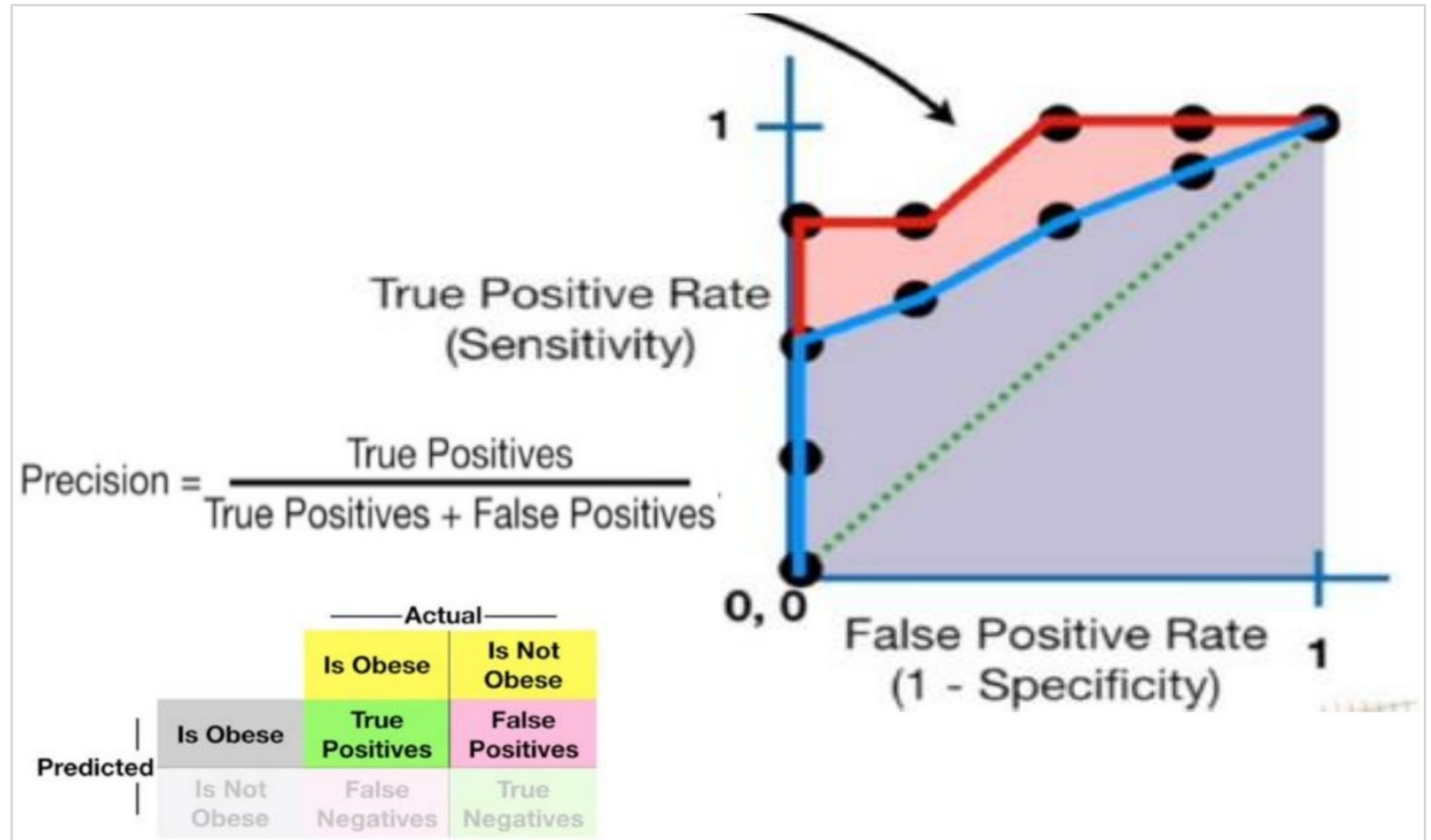


Precision Instead of FPR: Precision–Recall

Precision ignores true negatives — when negatives dominate, it shows how the positive class really fares

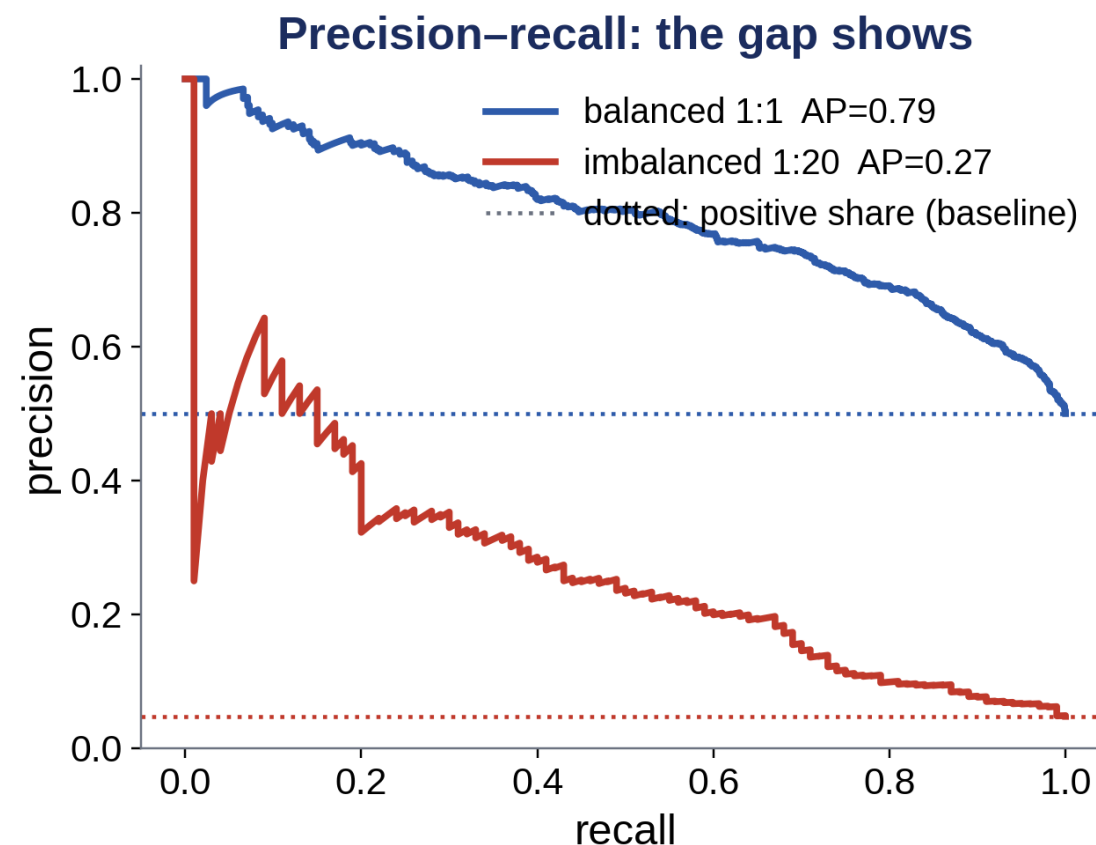
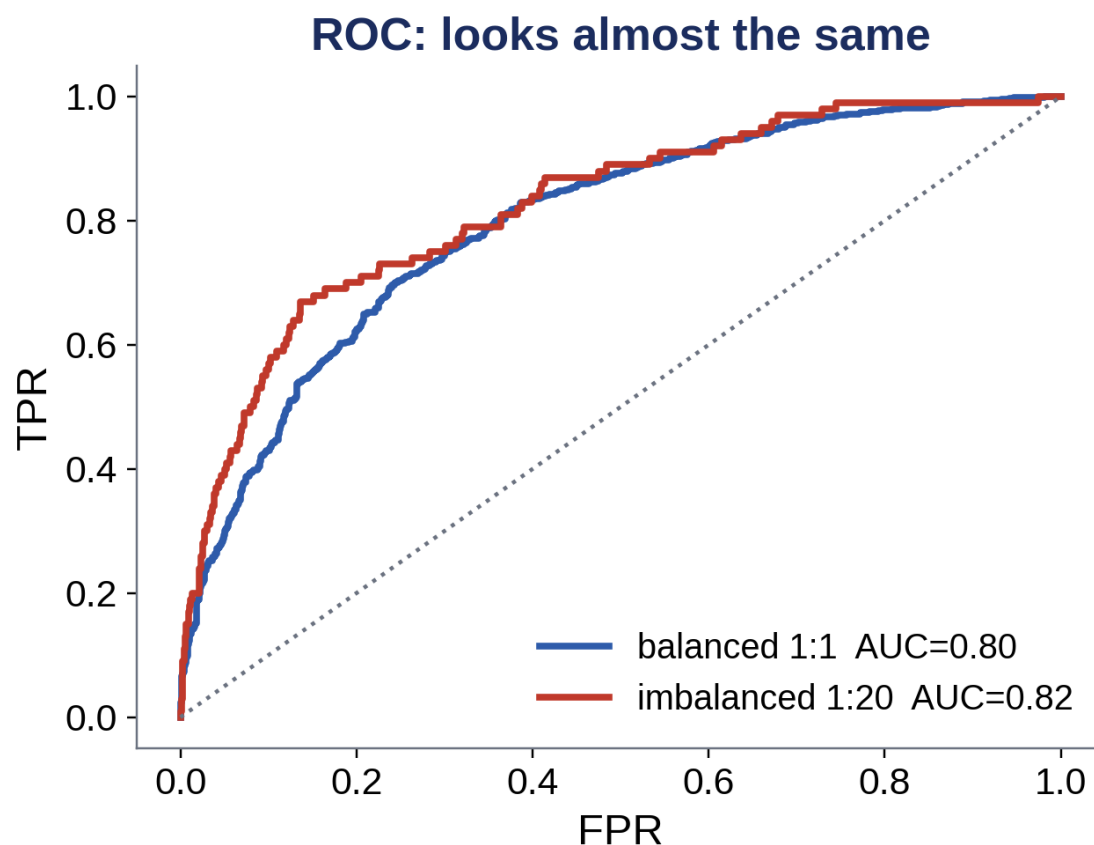
Correction

- 'Unaffected by imbalance' is wrong
- Precision depends on positive share
- PR baseline = positive share



Imbalanced Data: ROC Is Blunt, PR Is Sharp

Same scores, 20× more negatives: ROC barely moves, the PR curve drops sharply (simulation)



06

SECTION 06

Logistic Regression: Basic Lab

Three iris species and the metric functions

Lab 1: Classifying Iris Species

Four features, three species (train 120 / test 30, random_state=1)

Python · Colab

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
lr = LogisticRegression(max_iter=1000)
lr.fit(X_train, y_train)
y_pred = lr.predict(X_test)
print('Accuracy: {:.2f}%'.format(
    accuracy_score(y_test, y_pred) * 100))
print(lr.coef_.shape) # (3, 4)
```

Output

```
Accuracy: 96.67%
(3, 4)
Confusion matrix: [[11 0 0] [0 12 1] [0 0 6]]
```

$$P(y = k \mid \mathbf{x}) = \frac{e^{\mathbf{w}'_k \mathbf{x} + b_k}}{\sum_{j=1}^K e^{\mathbf{w}'_j \mathbf{x} + b_j}}$$

Correction

- Multiclass default is softmax (multinomial), not one-vs-rest — since sklearn 0.22
- Three rows = one score equation per species
- The three probabilities sum to 1

Lab 2: Metric Functions

All in `sklearn.metrics` — the argument order is always (actual, predicted)

Python · Colab

```
from sklearn.metrics import (accuracy_score,
                             precision_score, recall_score, f1_score,
                             confusion_matrix, classification_report,
                             roc_auc_score)

accuracy_score(y_test, y_pred)
precision_score(y_test, y_pred)
recall_score(y_test, y_pred)
f1_score(y_test, y_pred)
confusion_matrix(y_test, y_pred) # rows = actual
roc_auc_score(y_test, proba)     # uses probabilities
# multiclass: set average='macro' etc.
```

Watch out

- Accuracy is the same either way round
- Swapping the arguments swaps precision and recall
- AUC needs `predict_proba`, not `predict`

Exercise: Accuracy and the Confusion Matrix

3 of 10 are wrong → error rate 0.3, accuracy 0.7. Now fill in the confusion matrix

Idx	y	y_hat
1	1	1
2	1	1
3	1	0
4	1	1
5	1	1
6	0	0
7	0	0
8	0	1
9	0	1
10	0	0

	Actual 1	Actual 0
Predicted 1	TP = 4	FP = 2
Predicted 0	FN = 1	TN = 3

Metric	Calculation	Value
Accuracy	$(4+3) / 10$	0.70
Precision	$4 / 6$	0.67
Recall	$4 / 5$	0.80
Specificity	$3 / 5$	0.60
F1	$2PR / (P+R)$	0.73

07

SECTION 07

KOSPI Direction Lab

Colab notebook: can this month's indicators call next month's KOSPI up or down?

Target: 1 If the KOSPI Rises Next Month

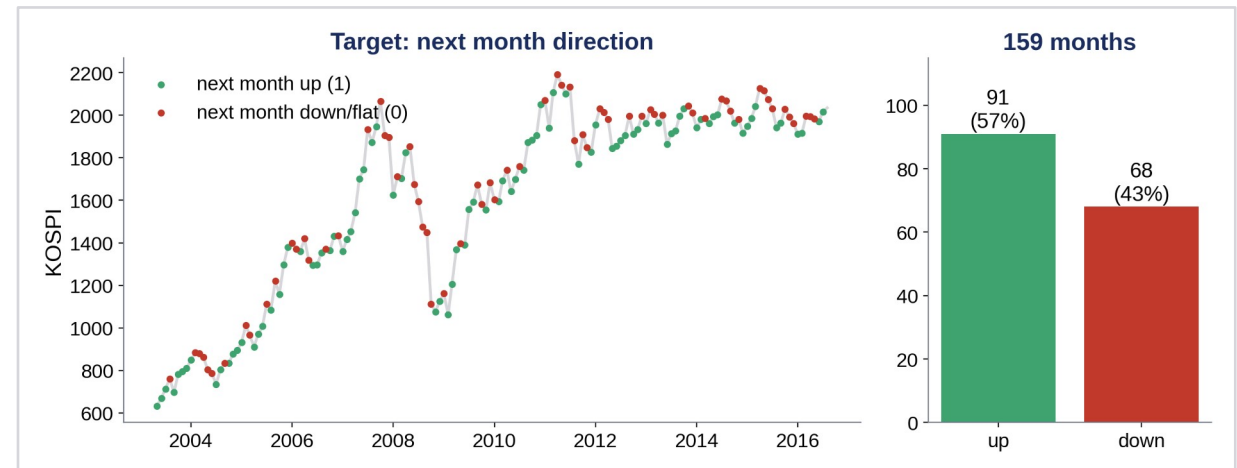
Each row pairs this month's 29 indicators with next month's direction — rows are independent

From the original notebook

```
kospi['next'] = kospi['KOSPI'].shift(-1)
kospi = kospi[:-1] # drop the last month
def classify(current, future):
    return 1 if future > current else 0
kospi['target'] = list(map(classify,
    kospi['KOSPI'], kospi['next']))
```

159 months

- Up 91 (57%) · down or flat 68 (43%)



Original Notebook Result: Accuracy 0.525

L1 logistic regression (liblinear, C = 10), random 75/25 split

From the original notebook

```
X_train, X_test, y_train, y_test = \
    train_test_split(X, y, random_state=42)
lr = LogisticRegression(solver='liblinear',
                        penalty='l1', C=10)
lr.fit(X_train, y_train)
accuracy_score(y_pred, y_test)    # 0.525
```

0.55

model accuracy
(40 test months)

0.50

share of up months
in the test set

Re-run from scratch

- The saved output (157 rows, 0.525) came from a state missing the first two rows
- A clean run gives 159 rows and accuracy 0.55

First question

- What is the baseline?
- Compare with 'always up'

Through the Confusion Matrix

Clean re-run (159 rows, random split, 40 test months)

	Actual up	Actual down
Predicted up	TP = 13	FP = 11
Predicted down	FN = 7	TN = 9

Metric	Value
Accuracy	0.550
Precision	0.542
Recall	0.650
F1	0.591
ROC AUC	≈ 0.55

Interpretation

- AUC 0.55: barely better than a coin flip (0.50)
- It calls 'up' a bit more often, which lifts recall
- With StandardScaler: accuracy 0.55, AUC 0.56 — same conclusion

Pitfall 1: Reading Accuracy Without a Baseline

The KOSPI rose in 57% of the 159 months — 'always up' gets that many right for free

0.550

logistic regression
(random split)

0.500

always up (random-split
test)

0.545

logistic regression
(mean, 100 splits)

0.570

always up (same 100
splits)

Baseline candidates

- Majority class (always up)
- Random guessing (AUC 0.5)

Principle

- Report model metrics next to the baseline
- With imbalance, add AUC and F1 to accuracy

Pitfall 2: L1 Regularization Without Standardizing

Unscaled features get unequal penalties — travelers (100,000s) vs. interest rates (a few %)

Python · Colab (fixed)

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
model = make_pipeline(
    StandardScaler(),
    LogisticRegression(solver='liblinear',
                      penalty='l1', C=10))
```

Why a pipeline

- In cross-validation the scaler is fit on training folds only

Why it matters

- L1 / L2 penalties act on coefficient size
- Large-unit features get small coefficients and escape the penalty
- scikit-learn 1.8+: `l1_ratio=1` instead of `penalty`

Monthly Index Direction Is Close to a Coin Flip

Repeat the random 75/25 split 100 times: accuracy averages 0.545, below always-up at 0.570

For 55% accuracy to mean anything, it must clear three hurdles

1 Beat the baseline

It must beat always-up (0.570 on the same splits) — it did in only 40 of 100 splits

2 Hold across other random splits

One split is one draw: over 100 splits accuracy ran from 0.375 to 0.725

3 Survive costs

A few points of hit rate vanish quickly after trading costs and taxes

Checklist for Evaluating a Classifier

The same order for logistic regression or any other classifier

#	Check	How
1	Computed the baseline first?	Majority-class share (DummyClassifier)
2	Checked more than one split?	Repeated random splits, RepeatedStratifiedKFold
3	Standardized the features?	Pipeline(StandardScaler, model)
4	Metric fits the purpose?	Recall, precision, F1, AUC, confusion matrix
5	Chose the threshold deliberately?	ROC / PR curves, error costs, Youden J
6	Chose C by cross-validation?	LogisticRegressionCV(cv=StratifiedKFold(5))

Discussion Questions

Q1

In the mouse example, threshold 0.9 had the highest accuracy. When should you still not use 0.9?

Threshold**Q2**

You receive a fraud model with AUC 0.95 on data where fraud is 0.1%. What else must you check?

Imbalance**Q3**

If a KOSPI-model coefficient gives $e^b = 1.3$, can you say 'when this indicator rises, the odds of an up month rise 30%'?

Interpretation**Q4**

What changes if you predict 'next month's return $\leq -5\%$ ' instead of direction?

Design

Summary

1

Logistic regression = a linear model for the log-odds

A sigmoid gives the probability; maximum likelihood (= minimum log loss) fits it. C is the inverse of regularization strength.

2

Threshold → confusion matrix → metrics

Precision, recall, F1 and specificity all come from the same four cells. ROC covers every threshold; AUC summarizes ranking.

3

Respect the baseline; check more than one split

The KOSPI direction model did not beat the baseline. With imbalanced data, look at the PR curve too.

Take-home: run the English Colab notebook and reproduce Section 07.

Corrections to the Original Material (1/2)

Page numbers refer to the original PDF (32. Logistic Regression, Korean)

Orig. page	Original	Corrected
2–3	Korean typos / $Y = a + b_1X_1 + \dots + X_m$	Typos fixed / last term $b_m \cdot X_m$ (coefficient missing)
4	'Use gradient descent' for a maximization problem	Gradient ascent on log-likelihood = descent on log loss; sklearn uses lbfgs etc.
22	'Precidted' typo; layout only mentioned	Stated that sklearn confusion_matrix has rows = actual, columns = predicted
25	Garbled table header; 'the average is still 50%'	Table restored; arithmetic mean 50.5%, accuracy 1%, $F1 \approx 2\%$ added
26	'F!', 'recal'	F1, recall; harmonic-mean comparison table added
33	Matrix labelled Infected / Not Infected	Read as Obese / Not Obese in this example (label in the source image)

Corrections to the Original Material (2/2)

Page numbers refer to the original PDF (32. Logistic Regression, Korean)

Orig. page	Original	Corrected
38	Receivr Operator Characteristic	Receiver Operating Characteristic
50, 56	'(075, 1)' and small typos	(0.75, 1); wording fixed
64–65	Typos (random forest, precision, calculation)	Fixed
65	'Precision ignores TN, so it is unaffected by imbalance'	Precision depends on the positive share; PR baseline = positive share (simulation added)
67	'3 classes, so one-vs-rest with 3 equations'	Current default is softmax (multinomial); the 3 rows are per-class score equations
Notebook	Saved output 157 rows / 0.525; a single random split; no scaling	Clean run 159 rows / 0.55; baselines, 100 repeated random splits, StandardScaler pipeline